

Платформа попутчиков Кыргызстан · Production Specification v2.3

Межгородские поездки · Telegram Mini App · Flutter · TypeScript

Версия 2.3 · Production-Ready · April 2026 · Full Roadmap

Параметр	Значение
Документ	Техническое задание v2.3
Назначение	Полная спецификация для старта разработки
Уровень	Production-ready, готов к передаче в команду
Стек	Node.js + Express (API) · Next.js (Web/Admin) · Vite+React (Mini App) · Flutter (iOS/Android)
База данных	PostgreSQL 16
Клиенты	5 клиентов: Mini App · Flutter · Web · Admin Panel · Telegram Bot
Рынок	Кыргызстан (межгород)
MVP срок	10-12 недель разработки
Команда	2 backend · 1 Flutter · 1 frontend · 0.5 QA · 0.5 DevOps
Design System	Nunito + Teal & Amber + Lucide Icons
Revision v2.3	Pre-booking чат + статус viewed + полный роадмап 3 этапа

Резюме для руководителя

Платформа попутчиков для межгородских поездок в Кыргызстане. Модель — peer-to-peer: водитель публикует поездку, пассажир бронирует место. Оплата в MVP — наличными напрямую водителю. Комиссия платформы и онлайн-оплата — Этап 3.

Ключевое отличие от конкурентов (inDrive, BlaBlaCar): гиперлокальная адаптация под КГ — Telegram Mini App как основной канал, двуязычный интерфейс (рус/кыр), локальные маршруты и платёжные методы на Этапе 3.

Ключевые решения v2.0

Решение	Обоснование
Backend на Express.js (отдельно от Next.js)	Next.js не подходит для WebSocket, cron jobs, долгих процессов
Mini App на Vite + React (не Next.js)	Mini App — тонкий клиент, SSR не нужен, Vite грузится в 3 раза быстрее
Без Redis и MinIO в MVP	Преждевременная оптимизация. Добавим на Этапе 2 по нагрузке
PostgreSQL без PostGIS	Для MVP достаточно простых FLOAT lat/lng полей
JWT access 15 мин + refresh 7 дней	Стандарт безопасности для мобильных приложений
SMS rate-limit: 1/мин, 5/сутки на номер	Защита от спама и разорения на SMS
Файлы на диске + Nginx (не MinIO)	MVP: диск + бэкап. Миграция на S3-совместимое — Этап 2
Bull Queue для фоновых задач	Нужен для OTP, cron, уведомлений. Использует Redis только на Этапе 2

Содержание

- 1** Контекст и цели проекта
- 2** Архитектура системы
- 3** Технологический стек
- 4** Инфраструктура и деплой
- 5** Модель данных (полная схема БД)
- 6** Роли и права доступа
- 7** Аутентификация и безопасность
- 8** Регистрация и онбординг
- 9** Верификация водителей
- 10** Публикация поездки — полный флоу
- 11** Поиск и бронирование — полный флоу
- 12** Управление запросами (водитель)
- 13** Чат между водителем и пассажиром
- 14** Система рейтингов и отзывов
- 15** Уведомления (Telegram Bot + Push)
- 16** Отмены, no-show и жалобы
- 17** Админ-панель (детально)
- 18** Операционная аналитика
- 19** API — полный справочник эндпоинтов
- 20** WebSocket события
- 21** Фоновые задачи и cron jobs
- 22** Обработка ошибок и слабого интернета
- 23** Локализация (русский + кыргызский)
- 24** Удаление аккаунта и GDPR/PDP
- 25** Telegram Bot — спецификация
- 26** Flutter приложение — особенности
- 27** Нефункциональные требования
- 28** План разработки и milestones
- 29** Критерии приёмки MVP (детально)
- 30** Риски и митигация

1 Контекст и цели проекта

1.1 Проблема которую решаем

Межгородские поездки в Кыргызстане (Бишкек—Ош, Бишкек—Каракол, Бишкек—Нарын, Бишкек—Иссык-Куль) сегодня организуются через разрозненные WhatsApp-группы, личные связи и стихийные стоянки-таксопарки у вокзалов. У пассажира нет возможности заранее проверить водителя, нет рейтингов, нет истории. У водителя нет канала найти попутчиков кроме соцсетей и "сарафана".

1.2 Решение

Цифровая платформа объединяющая водителей с свободными местами и пассажиров на межгородских маршрутах КГ. Водитель публикует поездку — пассажир бронирует место. Оплата наличными при встрече. Доверие — через верификацию документов, двусторонние рейтинги, чат внутри платформы.

1.3 Границы MVP (что включено, что не включено)

☑ Включено в MVP	☐ НЕ включено в MVP
Регистрация через Telegram + SMS OTP	Онлайн-оплата (Mbank/O!Деньги)
Верификация водителей (ручная админом)	Эскроу платежей
Публикация и поиск поездок	Доставка посылок
Бронирование с принятием/отклонением	Внутригородские поездки (такси-режим)
Чат между водителем и пассажиром	Комиссия платформы
Двусторонний рейтинг после поездки	Реферальная программа
Уведомления через Telegram Bot	Корпоративные аккаунты
Админ-панель (верификация, модерация)	Интеграция с API госорганов (МВД, права)
Операционный дашборд (метрики)	Анализ поведения пользователей (heatmaps)

1.4 Первый рынок

Запуск на трёх маршрутах одновременно: Бишкек↔Ош, Бишкек↔Каракол, Бишкек↔Нарын. После достижения 100 активных водителей на каждом маршруте — расширение на Бишкек↔Талас, Бишкек↔Чолпон-Ата, Бишкек↔Джалал-Абад.

1.5 Метрики успеха MVP

Метрика	Цель за 3 месяца после запуска
Зарегистрированных пассажиров	3000+
Верифицированных водителей	200+
Опубликованных поездок в неделю	400+
Завершённых поездок в неделю	300+
Процент принятия бронирований	>70%
Средний рейтинг водителей	>4.5
Retention Day-7	>30%

Метрика	Цель за 3 месяца после запуска
Retention Day-30	>15%

2 Архитектура системы

2.1 Общая схема



2.2 Разделение backend на два процесса

API Server и WebSocket Server — это два отдельных Node.js процесса. Причина: API stateless и масштабируется горизонтально добавлением инстансов, WebSocket stateful (хранит открытые соединения в памяти) и требует sticky sessions. На MVP оба процесса на одном сервере, на Этапе 2 — разные серверы.

2.3 Почему именно такая архитектура

Компонент	Почему именно так
Express, не Next.js для API	Next.js API routes не поддерживают WebSocket корректно, плохо с cron jobs и долгими процессами. Express — стандарт для Node backend.

Компонент	Почему именно так
Vite + React для Mini App	Mini App — тонкий клиент. SSR не нужен (в Telegram нет SEO). Vite собирает бандл в 3 раза меньше чем Next.js, что критично для 4G в регионах.
Next.js для Web и Admin	Публичный сайт требует SEO (SSR/ISR), Admin требует быстрой маршрутизации и удобного DX. Next.js идеален для обоих.
Flutter для мобильного	Один код для iOS и Android, производительность близкая к нативу. Для платформы с реальной навигацией и картами это важно.
PostgreSQL без PostGIS	На MVP не делаем geospatial queries типа "найди поездку в радиусе 5 км". Добавим на Этапе 2 когда появится точка-в-точку.
Без Redis в MVP	JWT stateless, кэш не нужен при нагрузке до 1k DAU. Добавим на Этапе 2 для сессий чата и rate limiting.
Без Kafka/RabbitMQ	Избыточно. На MVP используем node-cron + простые in-memory очереди через Bull-compatible библиотеку.

3 Технологический стек

3.1 Backend

Компонент	Технология	Версия	Назначение
Runtime	Node.js	20 LTS	Основная среда выполнения
Язык	TypeScript	5.4+	Строгая типизация (strict: true)
HTTP фреймворк	Express.js	4.19+	REST API сервер
ORM	Prisma	5.x	Типизированные запросы + миграции
БД	PostgreSQL	16	Основное хранилище
WebSocket	Socket.IO	4.7+	Real-time события и чат
Валидация	Zod	3.x	Схемы валидации запросов
Auth	jsonwebtoken	9.x	JWT подпись и верификация
Password hash	bcrypt	5.x	Хеш паролей администраторов
Логирование	Pino	9.x	Structured JSON logs
Cron	node-cron	3.x	Плановые задачи (автозаккрытие, напоминания)
HTTP client	axios	1.x	Запросы к внешним API (Telegram, SMS)
Telegram Bot	node-telegram-bot-api	0.66+	Отправка сообщений через Bot API
Testing	Vitest	1.x	Unit + integration тесты
E2E Testing	Supertest	7.x	HTTP E2E тесты

3.2 Клиенты

Клиент	Стек	Build tool	Ключевые библиотеки
Telegram Mini App	React 18 + TS	Vite 5	Zustand, react-hook-form, TanStack Query, i18next, Tailwind
Flutter iOS/Android	Flutter 3.x + Dart	Flutter CLI	Riverpod, go_router, dio, hive (offline), easy_localization
Web (публичный)	Next.js 14 + TS	Next	App Router, Tailwind, TanStack Query
Admin Panel	Next.js 14 + TS	Next	App Router, Tailwind, Recharts, shadcn/ui

3.3 Инфраструктура

Компонент	Инструмент	Назначение
Контейнеризация	Docker + docker-compose	Одинаковое окружение dev/prod
Reverse proxy	Nginx	SSL termination, отдача статики, проху на Node
SSL	Let's Encrypt + Certbot	Автоматическое обновление сертификатов
CI/CD	GitLab CI/CD	Автотесты + автодеплой
Мониторинг	Uptime Kuma	Проверка доступности сервисов (self-hosted)
Логи	Nginx access + app logs	Ротация logrotate, хранение 30 дней
Бэкап БД	pg_dump + cron	Ежедневно, хранение 30 дней

Компонент	Инструмент	Назначение
Карты	OpenStreetMap + Leaflet	Без биллинга Google Maps
SMS провайдер	Mega.kg или Nikita	Отправка OTP (локальный КГ провайдер)
Облако файлов	Локальный диск (MVP)	Этап 2: миграция на S3-совместимое

4 Инфраструктура и деплой

4.1 Требования к серверу (MVP)

Параметр	MVP	Этап 2 (после 1000 DAU)
CPU	4 vCPU	8 vCPU
RAM	8 GB	16 GB
SSD	160 GB	320 GB (или S3)
Сетевой трафик	Unlimited или 2 TB/мес	5 TB/мес
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Провайдер	DigitalOcean / Hetzner / Kloud.kg	—
Ориентировочная стоимость	\$40–60/мес	\$120–200/мес

4.2 Структура окружений

Окружение	Назначение	Домен	Кто имеет доступ
local	Разработка на ноутбуке	localhost	Разработчик
staging	Тестирование перед продом	staging.popytchik.kg	Вся команда
production	Боевое окружение	popytchik.kg	DevOps + лиды

4.3 Поддомены

Поддомен	Сервис	Клиент
api.popytchik.kg	API Server + WebSocket	Все клиенты
popytchik.kg	Публичный Web (Next.js)	Пользователи через браузер
app.popytchik.kg	Telegram Mini App (Vite build)	Telegram WebApp
admin.popytchik.kg	Admin Panel	Администраторы
files.popytchik.kg	Статические файлы (аватары, документы)	Все клиенты (через Nginx)

4.4 CI/CD пайплайн

```
stages:
  - lint      # ESLint, Prettier, TypeScript check
  - test      # Unit + Integration (Vitest)
  - build     # Docker images
  - deploy-staging # auto on main
  - e2e       # E2E тесты на staging
  - deploy-prod  # manual approve

Правила:
- Пуш в feature-branch → lint + test
- Merge в main         → staging deploy (автоматически)
- Tag v*.*.*           → production deploy (после ручного approve)
- Rollback              → redeploy предыдущего тэга (одна команда)
```

4.5 Бэкап и восстановление

Что бэкапим	Частота	Хранение	Как восстанавливать
PostgreSQL (pg_dump)	Ежедневно в 03:00	30 дней	pg_restore из нужного дампа
Файлы (документы, аватары)	Ежедневно rsync	30 дней	rsync обратно на диск
Код	В Git (GitLab)	Навсегда	git clone + checkout нужного тэга
.env секреты	Password manager	Навсегда	Копирование вручную

5 Модель данных

5.1 Принципы

- Все PK — UUID v4 (безопасность, не выдают порядок создания).
- Все timestamps в UTC, конвертация в UTC+6 на клиенте.
- Soft delete через поле deleted_at (NULL = не удалён).
- Индексы на все FK и часто фильтруемые поля.
- ENUM через PostgreSQL native ENUM или CHECK constraints.

5.2 Таблица users

Поле	Тип	Constraints	Описание
id	UUID	PK, DEFAULT gen_random_uuid()	Первичный ключ
telegram_id	BIGINT	UNIQUE NULL	Telegram User ID (если входил через TG)
phone	VARCHAR(20)	UNIQUE NOT NULL	Телефон в формате +996XXXXXXXXX
phone_verified_at	TIMESTAMPTZ	NULL	Когда был подтверждён OTP
name	VARCHAR(100)	NOT NULL	Имя пользователя
avatar_url	VARCHAR(500)	NULL	Путь к файлу аватара
roles	TEXT[]	DEFAULT '{passenger}'	Массив: passenger, driver
rating	DECIMAL(3,2)	DEFAULT 0.00	Средний рейтинг 0.00–5.00
rating_count	INTEGER	DEFAULT 0	Количество оценок
language	VARCHAR(2)	DEFAULT 'ru' CHECK(IN ('ru','ky'))	Язык интерфейса
is_blocked	BOOLEAN	DEFAULT FALSE	Заблокирован администратором
blocked_reason	TEXT	NULL	Причина блокировки
last_seen_at	TIMESTAMPTZ	NULL	Последняя активность
created_at	TIMESTAMPTZ	DEFAULT NOW()	Дата регистрации
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Последнее обновление
deleted_at	TIMESTAMPTZ	NULL	Soft delete

Индексы users

idx_users_phone (UNIQUE) · idx_users_telegram_id (UNIQUE WHERE NOT NULL) · idx_users_deleted_at (WHERE NULL)

5.3 Таблица driver_profiles

Поле	Тип	Constraints	Описание
id	UUID	PK	—
user_id	UUID	FK users.id UNIQUE NOT NULL	1:1 с users

Поле	Тип	Constraints	Описание
car_make	VARCHAR(50)	NOT NULL	Toyota, Honda, etc.
car_model	VARCHAR(50)	NOT NULL	Camry, Civic, etc.
car_year	SMALLINT	CHECK(1980..2030)	Год выпуска
car_color	VARCHAR(30)	NOT NULL	Цвет
car_plate	VARCHAR(15)	UNIQUE NOT NULL	Госномер
seats_count	SMALLINT	CHECK(1..7) NOT NULL	Количество пассажирских мест
license_photo_path	VARCHAR(500)	NOT NULL	Путь к фото прав
car_passport_path	VARCHAR(500)	NOT NULL	Путь к фото техпаспорта
car_photo_path	VARCHAR(500)	NOT NULL	Путь к фото автомобиля
selfie_path	VARCHAR(500)	NOT NULL	Путь к селфи
verification_status	VARCHAR(20)	CHECK(IN (...))	pending/verified/rejected/suspended/blocked
rejection_reason	TEXT	NULL	Причина отклонения
verified_at	TIMESTAMPZ	NULL	—
verified_by	UUID	FK users.id NULL	Администратор
total_trips	INTEGER	DEFAULT 0	Счётчик завершённых поездок
cancellations_30d	INTEGER	DEFAULT 0	Отмены за последние 30 дней
created_at	TIMESTAMPZ	DEFAULT NOW()	—
updated_at	TIMESTAMPZ	DEFAULT NOW()	—

Индексы driver_profiles

idx_dp_user_id (UNIQUE) · idx_dp_status · idx_dp_plate (UNIQUE)

5.4 Таблица trips

Поле	Тип	Constraints	Описание
id	UUID	PK	—
driver_id	UUID	FK users.id NOT NULL	Водитель
origin_city	VARCHAR(50)	NOT NULL	Город отправления
destination_city	VARCHAR(50)	NOT NULL	Город назначения
origin_address	VARCHAR(200)	NOT NULL	Точка отправления (текст)
origin_lat	DOUBLE PRECISION	NULL	Координата (Этап 2)
origin_lng	DOUBLE PRECISION	NULL	—
waypoints	JSONB	DEFAULT '[]'	Промежуточные остановки [{city, address}]
departure_at	TIMESTAMPTZ	NOT NULL	Дата и время отправления (UTC)
departure_flexible	BOOLEAN	DEFAULT FALSE	±30 минут
estimated_duration_min	INTEGER	NOT NULL	Оценочная длительность (для автозакрытия)
seats_total	SMALLINT	CHECK(1..7) NOT NULL	Всего мест
seats_available	SMALLINT	CHECK(>=0) NOT NULL	Свободных мест
price_per_seat	INTEGER	CHECK(>=50) NOT NULL	Сом
price_negotiable	BOOLEAN	DEFAULT FALSE	"Готов обсудить"
luggage	VARCHAR(10)	CHECK(IN ('yes','small','no'))	Багаж
preferences	JSONB	DEFAULT '{}'	{silence, music, no_smoking}
comment	VARCHAR(300)	NULL	Комментарий водителя
status	VARCHAR(20)	CHECK(IN (...))	active/completed/cancelled
cancelled_reason	TEXT	NULL	Если отменена водителем
idempotency_key	VARCHAR(100)	UNIQUE NULL	Защита от дублирования
created_at	TIMESTAMPTZ	DEFAULT NOW()	—
updated_at	TIMESTAMPTZ	DEFAULT NOW()	—
version	INTEGER	DEFAULT 1	Оптимистичная блокировка

Индексы trips

idx_trips_driver_id · idx_trips_status · idx_trips_departure_at · idx_trips_search (origin_city, destination_city, departure_at, status) — композитный для поисковых запросов

5.5 Таблица bookings

Поле	Тип	Constraints	Описание
id	UUID	PK	—
trip_id	UUID	FK trips.id NOT NULL	—
passenger_id	UUID	FK users.id NOT NULL	—

Поле	Тип	Constraints	Описание
seats_count	SMALLINT	CHECK(1..4) NOT NULL	Кол-во мест
comment	VARCHAR(300)	NULL	—
status	VARCHAR(25)	CHECK(IN ...)	pending/accepted/rejected/cancelled/no_show/completed/expired
cancelled_by	VARCHAR(10)	NULL CHECK(driver/passenger)	Кто отменил
cancelled_at	TIMESTAMPTZ	NULL	—
expires_at	TIMESTAMPTZ	NULL	Время автоистечения запроса
created_at	TIMESTAMPTZ	DEFAULT NOW()	—
updated_at	TIMESTAMPTZ	DEFAULT NOW()	—

UNIQUE constraint

UNIQUE(trip_id, passenger_id) WHERE status IN (pending, accepted) — пассажир не может иметь два активных бронирования на одну поездку

5.6 Таблица messages

Поле	Тип	Constraints	Описание
id	UUID	PK	—
booking_id	UUID	FK bookings.id NOT NULL	Привязка к брони
sender_id	UUID	FK users.id NOT NULL	—
text	VARCHAR(2000)	NOT NULL	Текст сообщения
is_read	BOOLEAN	DEFAULT FALSE	—
read_at	TIMESTAMPTZ	NULL	—
created_at	TIMESTAMPTZ	DEFAULT NOW()	—

Индексы messages

idx_messages_booking_id · idx_messages_created_at

5.7 Таблица ratings

Поле	Тип	Constraints	Описание
id	UUID	PK	—
trip_id	UUID	FK trips.id NOT NULL	—
rater_id	UUID	FK users.id NOT NULL	Кто оценивает
ratee_id	UUID	FK users.id NOT NULL	Кого оценивают
score	SMALLINT	CHECK(1..5) NOT NULL	Оценка
tags	TEXT[]	DEFAULT '{}'	Массив тегов
comment	VARCHAR(500)	NULL	Отзыв
created_at	TIMESTAMPTZ	DEFAULT NOW()	—

UNIQUE

UNIQUE(trip_id, rater_id, ratee_id) — одна оценка одному участнику за одну поездку

5.8 Таблицы поддержки

notifications — очередь уведомлений

Поле	Тип	Описание
id	UUID PK	—
user_id	UUID FK users.id	Получатель
type	VARCHAR(50) NOT NULL	booking_accepted, new_request, etc.
channel	VARCHAR(10)	CHECK(IN ('telegram','push','both'))
payload	JSONB	Данные для рендера
delivered	BOOLEAN DEFAULT FALSE	Отправлено
read_at	TIMESTAMPTZ NULL	Прочитано
created_at	TIMESTAMPTZ DEFAULT NOW()	—

complaints — жалобы пользователей

Поле	Тип	Описание
id	UUID PK	—
reporter_id	UUID FK users.id	Кто жалуется
target_user_id	UUID FK users.id NULL	На кого
target_trip_id	UUID FK trips.id NULL	На какую поездку
category	VARCHAR(50)	безопасность/мошенничество/грубость/no_show/другое
description	TEXT NOT NULL	Описание
status	VARCHAR(20)	new/in_review/resolved/dismissed
resolution	TEXT NULL	Решение администратора
handled_by	UUID FK users.id NULL	Администратор
created_at	TIMESTAMPTZ DEFAULT NOW()	—
resolved_at	TIMESTAMPTZ NULL	—

otp_codes — OTP для SMS верификации

Поле	Тип	Описание
id	UUID PK	—
phone	VARCHAR(20) NOT NULL	—
code_hash	VARCHAR(100) NOT NULL	bcrypt хеш кода (не храним открыто)
attempts	SMALLINT DEFAULT 0	Попытки ввода
expires_at	TIMESTAMPTZ NOT NULL	Через 5 минут
used_at	TIMESTAMPTZ NULL	Когда был использован
created_at	TIMESTAMPTZ DEFAULT NOW()	—

refresh_tokens — активные сессии

Поле	Тип	Описание
id	UUID PK	—
user_id	UUID FK users.id	—
token_hash	VARCHAR(100) NOT NULL	Хеш refresh токена
device_info	VARCHAR(300)	User-Agent или название устройства
expires_at	TIMESTAMP TZ NOT NULL	Через 7 дней
revoked_at	TIMESTAMP TZ NULL	При logout
created_at	TIMESTAMP TZ DEFAULT NOW()	—

cities — справочник городов КГ

Поле	Тип	Описание
id	SERIAL PK	—
name_ru	VARCHAR(100) NOT NULL	—
name_ky	VARCHAR(100) NOT NULL	—
region	VARCHAR(50)	Область
lat	DOUBLE PRECISION	—
lng	DOUBLE PRECISION	—
is_active	BOOLEAN DEFAULT TRUE	—

admin_actions — аудит действий администраторов

Поле	Тип	Описание
id	UUID PK	—
admin_id	UUID FK users.id	—
action	VARCHAR(50) NOT NULL	verify_driver, block_user, etc.
target_id	UUID NULL	ID объекта действия
target_type	VARCHAR(30) NULL	user, driver, trip, complaint
details	JSONB	Детали действия
ip_address	INET	IP администратора
created_at	TIMESTAMP TZ DEFAULT NOW()	—

6 Роли и права доступа

6.1 Роли

Роль	Где хранится	Как получает роль
passenger	users.roles[]	При регистрации автоматически
driver	users.roles[]	После верификации документов
admin	Отдельная таблица admins	Создаёт суперадмин
superadmin	Отдельная таблица admins	Первый создаётся вручную через SQL миграцию

6.2 Создание первого суперадмина

Первый суперадмин создаётся миграцией Prisma. Email и временный пароль задаются через переменные окружения SUPERADMIN_EMAIL и SUPERADMIN_TEMP_PASSWORD. После первого входа система требует смены пароля. После этого суперадмин создаёт остальных администраторов через Admin Panel.

6.3 Матрица прав

Действие	passenger	driver	admin	superadmin
Регистрация	☐	☐	☐	☐
Публикация поездки	☐	☐	☐	☐
Бронирование места	☐	☐	☐	☐
Чат	☐	☐	☐	☐
Отзывы	☐	☐	☐	☐
Просмотр всех поездок	☐	☐	☐	☐
Верификация водителей	☐	☐	☐	☐
Блокировка пользователя	☐	☐	Временная	Постоянная + временная
Управление городами	☐	☐	☐	☐
Создание администраторов	☐	☐	☐	☐
Просмотр аудит-логов	☐	☐	Свои	Все
Экспорт данных	☐	☐	Ограниченный	Полный

7 Аутентификация и безопасность

7.1 Главный принцип

Телефон — операционный идентификатор, не просто логин

Водитель и пассажир связываются по телефону. Поэтому номер верифицируется через SMS при регистрации у всех без исключения — даже если Telegram, Google или Apple уже передали номер. Данные от OAuth провайдера используются только как подсказка в поле ввода, не как истина. SMS подтверждение гарантирует что номер реально у человека прямо сейчас.

7.2 Способы входа

Провайдер	Клиент	Механизм	SMS при регистрации?
Telegram	Mini App	initData HMAC-SHA256 подпись через bot_token	Да — всегда
Google	Flutter / Web	OAuth 2.0 → id_token верификация через Google JWKS	Да — всегда
Apple	Flutter iOS / Web	Sign In with Apple → identity_token верификация	Да — всегда
Phone + Password	Все клиенты	Номер телефона + пароль (bcrypt rounds=12)	Да — при регистрации
Email + Password + 2FA	Admin Panel only	Email + bcrypt + TOTP (Google Authenticator)	Нет

Правило повторного входа

SMS OTP нужен ТОЛЬКО при первой регистрации. При повторном входе через любой провайдер SMS не нужен никогда — пользователь уже доказал владение номером при регистрации.

7.3 JWT токены — полная спецификация

Параметр	Access Token	Refresh Token
Время жизни	15 минут	Продлевается при каждом активном использовании
Разлогин	Через 15 мин без refresh	Только если не заходил более 30 дней подряд
Алгоритм	HS256	HS256
Секрет	JWT_ACCESS_SECRET (env)	JWT_REFRESH_SECRET (env)
Хранение Mini App	Memory (JS variable)	sessionStorage (очищается при закрытии)
Хранение Flutter	Memory (Riverpod state)	flutter_secure_storage (Keychain/Keystore)
Хранение Web	Memory (React state)	HttpOnly + Secure + SameSite=Strict cookie
Хранение сервер	Не хранится (stateless)	SHA-256 хеш в таблице refresh_tokens
Revocation	Не нужен (15 мин TTL)	revoked_at в БД + rotation при каждом /refresh
Несколько устройств	Независимы	Каждое устройство — своя строка в refresh_tokens

7.4 Payload access токена

```
{
  "sub":      "uuid-пользователя",
  "phone":    "+996700123456",
  "roles":    ["passenger"],      // passenger | driver | both | admin | superadmin
  "provider": "telegram",        // telegram | google | apple | phone
  "iat":      1713445200,
  "exp":      1713446100         // iat + 15 минут
}
```

ЗАПРЕЩЕНО класть в токен:

- rating, is_blocked (проверяются на сервере при каждом запросе)
- name, avatar_url (персональные данные, не нужны в токене)
- password_hash (очевидно)

7.5 Token Reuse Detection — защита от кражи

Механизм обнаружения кражи refresh токена

При каждом POST /auth/refresh старый токен помечается как использованный (used_at = NOW()). Если кто-то попытался использовать уже использованный refresh токен — это признак кражи. Все refresh токены пользователя немедленно инвалидируются (revoked_at = NOW()), на телефон приходит уведомление через Telegram Bot.

```
async function refreshTokens(incomingToken) {
  const hash = sha256(incomingToken);
  const record = await db.refresh_tokens.findOne({ token_hash: hash });
  if (!record) throw new UnauthorizedException("TOKEN_NOT_FOUND");
  if (record.used_at !== null) {
    // Токен уже был использован — детектируем кражу
    await db.refresh_tokens.updateMany(
      { user_id: record.user_id },
      { revoked_at: new Date() } // инвалидируем ВСЕ сессии
    );
    await notifyUser(record.user_id, "security_alert_token_reuse");
    throw new UnauthorizedException("TOKEN_REUSE_DETECTED");
  }
  if (record.revoked_at || record.expires_at < new Date())
    throw new UnauthorizedException("TOKEN_EXPIRED_OR_REVOKED");
  // Rotation: помечаем старый использованным, создаём новый
  await db.refresh_tokens.update({ id: record.id }, { used_at: new Date() });
  const newRefresh = generateSecureToken();
  await db.refresh_tokens.create({
    user_id: record.user_id,
    token_hash: sha256(newRefresh),
    device_info: record.device_info,
    expires_at: addDays(new Date(), 30),
  });
  return { accessToken: signJWT(...), refreshToken: newRefresh };
}
```

7.6 Rate limiting

Эндпоинт	Лимит	Ключ	Причина
POST /auth/phone/send-otp	1 запрос в минуту	phone номер	SMS стоят денег
POST /auth/phone/send-otp	Макс 5 SMS в сутки	phone номер	Защита от разорения
POST /auth/phone/verify	5 попыток на OTP	otp_codes.id	Брутфорс
POST /auth/phone/verify	Блок 15 мин после 5 ошибок	phone номер	Брутфорс
POST /auth/telegram	10 запросов в минуту	IP адрес	Спам
POST /auth/google	10 запросов в минуту	IP адрес	Спам
POST /auth/apple	10 запросов в минуту	IP адрес	Спам
POST /auth/refresh	30 запросов в минуту	user_id	Аномальная активность
POST /trips	5 поездок в час	driver user_id	Флуд
POST /bookings	20 бронирований в час	passenger user_id	Спам
POST /complaints	3 жалобы в час	user_id	Злоупотребление
Все эндпоинты	100 запросов в минуту	IP адрес	Anti-DDoS

7.7 Видимость номера телефона

Защита от обхода платформы

Основной риск: пользователи договорятся напрямую и будут ездить без платформы. Решение — скрывать номера до принятия бронирования.

Состояние booking	Пассажир видит номер водителя?	Водитель видит номер пассажира?
До бронирования	НЕТ — только имя и рейтинг	НЕТ — только имя и рейтинг
status = pending	НЕТ	НЕТ
status = accepted	ДА — полный номер	ДА — полный номер
status = rejected / expired	НЕТ — снова скрыт	НЕТ — снова скрыт
status = cancelled	НЕТ — снова скрыт	НЕТ — снова скрыт
status = completed	ДА — ещё 48 часов	ДА — ещё 48 часов
Чат до accepted	Фильтр regex: [номер скрыт]	Фильтр regex: [номер скрыт]

7.8 Защита от типичных атак

Атака	Защита
SQL Injection	Prisma ORM — все запросы параметризованы. Raw queries запрещены в code review.

Атака	Защита
XSS	React/Next автоматически экранируют. ESLint правило: запрет dangerouslySetInnerHTML.
CSRF	JWT в Authorization header для Mini App/Flutter. Web — SameSite=Strict + проверка Origin header.
Brute force OTP	Rate limit + блокировка номера на 15 минут после 5 попыток. attempts++ при каждой ошибке.
Token theft	Token Reuse Detection: повторный refresh → инвалидация всех сессий + уведомление пользователю.
Replay attack	idempotency_key на критичных операциях. Каждый refresh token — одноразовый.
Загрузка вредоносных файлов	Валидация MIME + расширение + magic bytes. Только image/jpeg, image/png. Max 5 MB.
Account takeover через смену номера	Смена номера требует re-auth через текущий провайдер + OTP на новый номер.
DDoS	Nginx rate limit на IP + Cloudflare на проде.
Enumeration через OTP	Одинаковый response time и ответ независимо от того существует номер или нет.
Session fixation	Refresh token rotation: каждый /refresh выдаёт новый token, старый инвалидируется.

8 Регистрация, онбординг и повторный вход

8.1 Регистрация — первый вход (общий принцип)

Независимо от выбранного провайдера после OAuth или ввода данных всегда появляется один и тот же экран — "Введите номер для связи". Если провайдер передал номер — он подставляется в поле автоматически, но пользователь может его изменить. Это гарантирует что номер реально у него в руках прямо сейчас.

8.2 Флоу регистрации через Telegram Mini App

- 1 **Telegram открывает Mini App**
initData подписан HMAC-SHA256 с bot_token. Срок действия initData — 24 часа.
- 2 **POST /auth/telegram с initData**
Сервер проверяет подпись. Если невалидно — 401.
- 3 **Сервер ищет пользователя по telegram_id**
Если найден И phone_verified_at IS NOT NULL — выдаёт JWT сразу (повторный вход, без SMS).
- 4 **Если не найден — создаёт нового**
roles = [passenger], telegram_id установлен, phone = NULL. Выдаётся provisional JWT.
- 5 **Редирект на экран ввода номера**
Если Telegram передал номер — подставляем как подсказку. Пользователь может изменить.
- 6 **POST /auth/phone/send-otp**
Rate limit: 1/мин, 5/сутки. Генерация кода, bcrypt(code) → otp_codes, TTL 10 мин.
- 7 **Пользователь вводит 6-значный код**
POST /auth/phone/verify. При ошибке — attempts++. После 5 — блок 15 мин.
- 8 **Если номер уже занят другим аккаунтом**
Предлагаем привязать Telegram к существующему аккаунту — не создаём дубль.
- 9 **Привязка номера к аккаунту**
users.phone = номер, users.phone_verified_at = NOW().
- 10 **Оptionальный пароль**
"Придумайте пароль как резервный способ входа". Кнопка "Пропустить" — напоминание в настройках.
- 1 **Выдача финального JWT**
1 Полноценный access + refresh token. Онбординг завершён.

8.3 Флоу регистрации через Flutter/Web (Phone + Password)

- 1 **Пользователь вводит +996XXX**
Валидация на клиенте: /^+996[0-9]{9}\$/. Если номер уже занят — предлагаем войти.
- 2 **POST /auth/phone/send-otp**
Проверка rate limit. Генерация OTP, bcrypt(code) → otp_codes, TTL 10 мин.
- 3 **Пользователь вводит 6-значный код**
POST /auth/phone/verify. После 5 ошибок — блок 15 мин.

- 4 **Установка пароля**
Обязательный экран: минимум 8 символов, 1 цифра. `bcrypt(password, 12) → users.password_hash`.
- 5 **Find or create пользователя**
`users.phone = номер, phone_verified_at = NOW(), roles = [passenger]`.
- 6 **Выдача JWT**
Полноценный access + refresh token.
- 7 **Первый вход — заполнение профиля**
Имя + аватар (опционально). Язык (ru | ky).

8.4 Заполнение профиля после регистрации

Поле	Обязательн о	Источник по умолчанию	Валидация
Имя	Да	Из Telegram/Google/Apple или пусто	2-50 символов
Аватар	Нет	Из провайдера или placeholder	JPEG/PNG, <5 MB
Язык	Да	Из Telegram language_code или ru	ru ky
Роль	Да	passenger	passenger driver (driver → верификация)

8.5 Повторный вход — второй раз и далее

Ключевой принцип UX: пользователь никогда не видит экран входа при регулярном использовании

Refresh token продлевается при каждом активном использовании. Разлогин происходит только если не заходил более 30 дней подряд. При повторном входе SMS не нужен никогда — один тап через провайдер и JWT выдан.

- 1 **Приложение открывается**
Клиент проверяет access token. Если живой — пользователь внутри без каких-либо экранов.
- 2 **Access token истёк**
Axios interceptor автоматически делает POST /auth/refresh. Пользователь не замечает.
- 3 **Refresh успешен**
Новый access + refresh token. Оригинальный запрос повторяется автоматически.
- 4 **Refresh истёк (30+ дней без входа)**
Показывается экран входа. Один тап или ввод пароля — новый JWT выдан.
- 5 **SMS при повторном входе**
Никогда не нужен — пользователь уже доказал владение номером при регистрации.

8.6 Deferred Action — отложенное действие

Проблема которую решаем

Пользователь без авторизации нашёл поездку и нажал "Забронировать место". Без deferred action — после входа он оказывается на главной и теряет контекст. С deferred action — после входа видит уже открытый экран бронирования, как будто разлогина не было.

- 1 **Неавторизованный пользователь нажимает защищённое действие**
"Забронировать место" / "Написать водителю" / "Оставить отзыв" / "Посмотреть контакты".
- 2 **Приложение сохраняет намерение в sessionStorage**
Ключ: `kosho_deferred_action`. Значение: `{action, данные действия, saved_at: Date.now()}`.
- 3 **Редирект на экран входа**
URL не меняется. Возврат к намерению — автоматически после входа.
- 4 **Пользователь проходит авторизацию**
Любой провайдер — регистрация или повторный вход.
- 5 **После получения JWT — проверка sessionStorage**
Читаем `kosho_deferred_action`. Проверяем: `saved_at + 15 мин > NOW()`.
- 6 **Если намерение живое — выполняем автоматически**
Редирект на нужный экран с заполненными данными. Пользователь видит готовую форму.
- 7 **Удаление намерения из sessionStorage**
После выполнения — удаляем запись. Один раз выполняется, не повторяется.
- 8 **Если намерение протухло (>15 мин)**
Пользователь попадает на главную. Toast: "Сессия истекла — найдите поездку снова".

Поддерживаемые действия в deferred action:

Действие	Данные в sessionStorage	Экран после входа
Забронировать место	<code>{action: book_trip, trip_id, seats, comment}</code>	Экран подтверждения бронирования
Написать водителю	<code>{action: open_chat, booking_id}</code>	Экран чата с водителем
Оставить отзыв	<code>{action: rate_user, trip_id, ratee_id}</code>	Экран оценки поездки
Посмотреть контакты	<code>{action: view_contacts, booking_id}</code>	Детали брони с контактами
Подать жалобу	<code>{action: file_complaint, target_user_id, target_trip_id}</code>	Форма жалобы

```
// Клиент – сохранение намерения перед редиректом
function deferAndLogin(action) {
  sessionStorage.setItem("kosho_deferred_action", JSON.stringify({
    ...action,
    saved_at: Date.now()
  }));
  router.push("/auth/login");
}

// Клиент – выполнение после успешного входа
function executeDeferredAction() {
  const raw = sessionStorage.getItem("kosho_deferred_action");
  if (!raw) return;
  const intent = JSON.parse(raw);
  const TTL = 15 * 60 * 1000; // 15 минут
  sessionStorage.removeItem("kosho_deferred_action");
  if (Date.now() - intent.saved_at > TTL) {
    router.push("/");
    toast.info("Сессия истекла. Найдите поездку снова.");
    return;
  }
}
```

```

}
switch (intent.action) {
  case "book_trip":
    router.push(`/trips/${intent.trip_id}/book?seats=${intent.seats}`);
    break;
  case "open_chat":
    router.push(`/bookings/${intent.booking_id}/chat`);
    break;
  case "rate_user":
    router.push(`/trips/${intent.trip_id}/rate/${intent.ratee_id}`);
    break;
  case "view_contacts":
    router.push(`/bookings/${intent.booking_id}?tab=contacts`);
    break;
  case "file_complaint":
    router.push(`/complaint?user=${intent.target_user_id}`);
    break;
  default:
    router.push("/");
}
}

```

8.7 Смена номера телефона

1 Настройки → Сменить номер

Доступно только авторизованному пользователю.

2 Подтверждение личности

Re-auth через текущий провайдер: Telegram silent re-auth / Google silent re-auth / ввод пароля.

3 Ввод нового номера

Валидация формата. Проверка UNIQUE — номер не занят другим аккаунтом.

4 ОТП на НОВЫЙ номер

SMS уходит на новый номер, не на старый. Rate limit применяется к новому номеру.

5 Верификация ОТП

Стандартная: 6 цифр, 5 попыток, TTL 10 минут.

6 Обновление номера

users.phone = новый номер. Все активные refresh токены инвалидируются.

7 Выдача нового JWT

С обновлённым phone в payload. Пользователь остаётся на текущем устройстве.

8 Уведомление

Telegram Bot на старый аккаунт: "Номер изменён. Если не вы — обратитесь в поддержку."

8.8 Восстановление доступа

Сценарий А — забыл пароль (телефон доступен):

```

"Забыл пароль" → ввод номера → POST /auth/phone/send-otp
→ ввод ОТП → POST /auth/phone/verify
→ экран нового пароля → PATCH /users/me/password
→ JWT выдан, пользователь внутри

```

Сценарий В — нет доступа ни к одному провайдеру:

"Войти только по телефону" → ввод номера → POST /auth/phone/send-otp
→ ввод OTP → экстренный доступ без пароля
→ система предлагает: "Привяжите провайдер или установите пароль"
→ пользователь может пропустить (напоминание в профиле)

8.9 Роли пользователей

Роль	Значение	Как получает	Переключение JWT
passenger	roles: ['passenger']	При регистрации автоматически	При регистрации
driver	roles: ['driver']	Только через both — нельзя быть только driver	После верификации
both	roles: ['passenger', 'driver']	Пассажир подаёт заявку на водителя	При следующем refresh после одобрения
admin	Таблица admins	Создаёт суперадмин	Отдельный JWT секрет
superadmin	Таблица admins	SQL миграция при первом деплое	Отдельный JWT секрет

Один аккаунт — одна история

Пассажир который хочет стать водителем НЕ создаёт новый аккаунт. Он подаёт заявку в своём профиле. После одобрения roles обновляется до [passenger, driver]. История поездок, рейтинг и все данные сохраняются.

8.10 Когда используется SMS — строгое правило

Сценарий	SMS нужен?	Объяснение
Первая регистрация (любой провайдер)	ДА — всегда	Единственная гарантия владения номером
Смена номера телефона	ДА — на новый номер	Верификация нового номера
Восстановление пароля	ДА — на текущий номер	Аутентификация через телефон
Экстренный вход без провайдеров	ДА — на текущий номер	Единственный способ доказать владение
Повторный вход (любой провайдер)	НЕТ — никогда	Провайдер уже верифицирован при регистрации
Обновление access токена (/refresh)	НЕТ — никогда	Автоматический фоновый процесс
Смена пароля (текущий известен)	НЕТ	Текущий пароль — достаточная аутентификация
Добавление нового OAuth провайдера	НЕТ	Пользователь уже авторизован

8.11 Новая таблица auth_providers

Изменение схемы БД

Для поддержки нескольких OAuth провайдеров на один аккаунт вводится таблица auth_providers вместо одиночного поля telegram_id в users.

Поле	Тип	Constraints	Описание
id	UUID	PK	—
user_id	UUID	FK users.id NOT NULL	Аккаунт пользователя
provider	VARCHAR(20)	CHECK(IN ('telegram','google','apple','phone'))	Тип провайдера
provider_user_id	VARCHAR(200)	NOT NULL	ID у провайдера (telegram_id, google sub, phone)
provider_data	JSONB	DEFAULT '{}'	email, name, picture от провайдера
created_at	TIMESTAMPTZ	DEFAULT NOW()	—

UNIQUE constraint

UNIQUE(provider, provider_user_id) — один провайдер-аккаунт только на один аккаунт платформы.

Новое поле в users	Тип	Назначение
password_hash	VARCHAR(100) NULL	bcrypt(password, 12). NULL если пароль не установлен.
terms_accepted_at	TIMESTAMPTZ NULL	Дата принятия пользовательского соглашения
notifications_enabled	BOOLEAN DEFAULT TRUE	Подписка на уведомления Telegram Bot

8.12 Новые API эндпоинты Auth

Метод	Путь	Описание	Auth
POST	/auth/google	Вход через Google (id_token)	Нет
POST	/auth/apple	Вход через Apple (identity_token)	Нет
POST	/auth/phone/login	Повторный вход: phone + password	Нет
POST	/auth/phone/send-otp	Отправить OTP	Нет
POST	/auth/phone/verify	Подтвердить OTP	Нет
POST	/auth/telegram	Вход через Telegram	Нет
POST	/auth/refresh	Обновить токены (rotation)	Refresh
POST	/auth/logout	Выйти (revoke текущего refresh)	JWT
POST	/auth/logout/all	Выйти со всех устройств	JWT
PATCH	/users/me/password	Установить / сменить пароль	JWT
PATCH	/users/me/phone	Сменить номер телефона	JWT
POST	/users/me/providers	Добавить OAuth провайдер	JWT
DELETE	/users/me/providers/:provider	Отвязать провайдер	JWT

9 Верификация водителей

9.1 Процесс верификации — детально

- 1 Экрaн данных об авто**
Марка, модель, год, цвет, госномер, количество пассажирских мест (1-7)
- 2 Экрaн фото прав**
Камера или галерея. Лицевая сторона. Валидация: min 800x600, макс 5MB, MIME image/jpeg или image/png
- 3 Экрaн фото техпаспорта**
Камера или галерея. Лицевая сторона с госномером, совпадающим с введенным
- 4 Экрaн фото авто**
Автомобиль спереди, госномер четко виден
- 5 Экрaн селфи**
Фронтальная камера предпочтительна. Лицо занимает не менее 40% кадра.
- 6 POST /drivers/verification**
Все файлы + данные. Транзакция: создание driver_profile со статусом pending.
- 7 Уведомление администратору**
Socket.IO push в Admin Panel + email если настроен
- 8 Ручная проверка администратора**
SLA 24 часа. Если >24 часа — escalation на суперадмина
- 9 Решение администратора**
VERIFY (approve) / REJECT (с причиной) / REQUEST_MORE_DOCS (конкретный запрос)
- 1 Уведомление водителю**
Telegram Bot сообщение + push. При verify — driver добавляется в users.roles

9.2 SLA и эскалация

Событие	SLA	Эскалация при нарушении
Верификация после подачи заявки	24 часа	Уведомление суперадмину через 24 часа
Повторная подача после REJECT	Нет ограничения	—
Ответ администратора на REQUEST_MORE_DOCS	48 часов после догрузки	Автоматический возврат в общую очередь

9.3 Автоматические проверки перед ручной

Проверка	При нарушении
Госномер не существует в car_plate (UNIQUE)	Ошибка формы "Этот номер уже зарегистрирован"
Все 4 фото загружены	Нельзя подать заявку
Фото >800x600 pixels	Ошибка "Загрузите более качественное фото"
MIME type image/jpeg или image/png	Ошибка формы
Размер файла <5 MB	Сжать автоматически на клиенте
Год выпуска авто в диапазоне 1980–текущий	Ошибка валидации

10.1 Бизнес-правила публикации

Правило	Детали
Только верифицированный водитель	<code>driver_profile.verification_status = verified</code>
Не более одной активной поездки в день на водителя	<code>UNIQUE(driver_id, departure_at::date) WHERE status = active</code>
Горизонт публикации	Не более чем на 60 дней вперёд, не ранее чем через 30 минут
Минимальная цена	50 сом за место (защита от спама)
Максимальная цена	10 000 сом за место (защита от ошибки)
<code>seats_total ≤ driver_profile.seats_count</code>	Нельзя предложить больше мест чем в машине
<code>idempotency_key</code> обязателен	UUID v4 с клиента, защита от двойного создания
<code>estimated_duration_min</code> рассчитывается автоматически	По таблице расстояний между городами (справочник)

10.2 Экраны публикации

Экран 1 — Маршрут

- Города из справочника `cities` (autocomplete), промежуточные остановки (до 3), точный адрес отправления

Экран 2 — Дата и время

- Date picker (не ранее чем +30 мин), time picker, чекбокс "Время может сдвинуться ±30 мин"

Экран 3 — Места и цена

- Stepper количества мест, input цены (подсказка средней по маршруту), toggle "Готов обсудить"

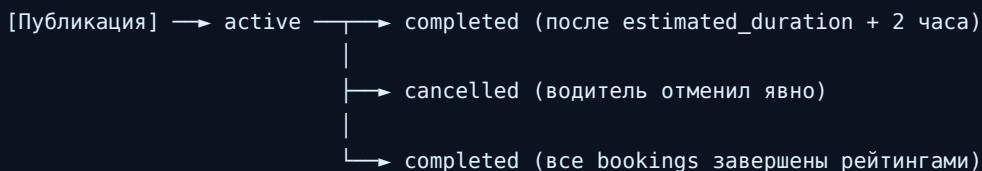
Экран 4 — Дополнительно

- Радио для багажа, мультиселект предпочтений, textarea комментария (до 300 символов)

Экран 5 — Предпросмотр

- Карточка в точности как увидят пассажиры. Кнопка "Опубликовать" → POST /trips с `idempotency_key`

10.3 Жизненный цикл поездки



Автозаккрытие: cron job каждые 15 минут ищет поездки где
`status = active AND departure_at + estimated_duration_min + 2h < NOW()`
 → `status = completed`, уведомление обоим сторонам с просьбой оценить

11.1 Параметры поиска

Параметр	Тип	Обязательн о	Валидация
from_city	string	Да	Есть в cities, is_active = true
to_city	string	Да	Есть в cities, != from_city
date	ISO date	Нет	От сегодня до +60 дней
seats	integer	Нет	1..4, по умолчанию 1
min_price	integer	Нет	>= 50
max_price	integer	Нет	>= min_price
only_verified	boolean	Нет	По умолчанию true
luggage	string	Нет	yes / small / no
sort	string	Нет	time (default) / price_asc / rating_desc
cursor	string	Нет	Cursor-based пагинация
limit	integer	Нет	1..50, по умолчанию 20

11.2 Флоу бронирования

- Пассажир видит детали поездки**
GET /trips/:id — полные данные + профиль водителя + недавние отзывы
- Нажимает "Забронировать"**
Форма: кол-во мест (1..seats_available), комментарий (до 300 символов)
POST /bookings
Транзакция: SELECT...FOR UPDATE на trips.id, проверка seats_available, создание booking со статусом pending, expires_at = NOW() + 1h
- Сервер отправляет Socket.IO событие водителю**
new_booking_request + Telegram Bot уведомление
- Пассажир видит статус "Ждём ответа"**
GET /bookings/:id возвращает статус + таймер до expires_at
- Водитель принимает**
PATCH /bookings/:id/accept — транзакция: проверка seats_available, seats_available -= seats_count, status = accepted, чат открыт
- Пассажир получает уведомление**
booking_accepted + Telegram Bot. Можно писать в чате.
- Альтернатива — отклонение**
PATCH /bookings/:id/reject — status = rejected, seats_available НЕ меняется
- Альтернатива — таймаут 1 час**
Cron job: status = expired для всех pending с expires_at < NOW()

11.3 Критичное: защита от race condition на seats_available

Проблема

Два пассажира одновременно бронируют последнее место. Без транзакции оба получат "accepted" — водитель получит двух пассажиров на одно место.

Решение — SELECT FOR UPDATE

При POST /bookings и PATCH /bookings/:id/accept транзакция включает SELECT trip FOR UPDATE. Это блокирует строку до конца транзакции. Вторая транзакция ждёт. Если seats_available стало 0 — возврат 409 Conflict.

```
// Псевдокод транзакции
BEGIN;
  SELECT * FROM trips WHERE id = $1 FOR UPDATE;
  IF seats_available < requested_seats THEN
    ROLLBACK;
    RETURN 409 Conflict "Мест больше нет";
  END IF;
  UPDATE trips SET seats_available = seats_available - $requested WHERE id = $1;
  INSERT INTO bookings (...) VALUES (...);
COMMIT;
```

12.1 Экран "Входящие запросы"

Элемент карточки	Источник данных
Фото пассажира	users.avatar_url
Имя	users.name
Рейтинг	users.rating (или "Новый" если rating_count < 3)
Количество мест	bookings.seats_count
Комментарий пассажира	bookings.comment
Время запроса	bookings.created_at ("X минут назад")
Таймер до автоистечения	bookings.expires_at - NOW()
Кнопка "Принять"	PATCH /bookings/:id/accept
Кнопка "Отклонить"	PATCH /bookings/:id/reject

12.2 Побочные эффекты принятия запроса

Действие	Изменения в системе
Транзакция акцепт	bookings.status = accepted, trips.seats_available -= N
Если seats_available стало 0	Все остальные pending запросы на эту поездку → status = expired
Открытие чата	Чат становится доступным (UI разблокируется)
Уведомление пассажиру	booking_accepted + Telegram Bot
Отмена других pending запросов пассажира	Если пассажир искал на эту же дату — остальные bookings в pending → cancelled_by_passenger автоматически (по флагу в пользовательских настройках)

12.3 Экран "Мои поездки" водителя

Вкладка	Фильтр	Сортировка
Активные	status = active AND departure_at > NOW()	По departure_at ASC
В пути	status = active AND departure_at <= NOW() < departure_at + duration	По departure_at ASC
Прошедшие	status = completed	По departure_at DESC
Отменённые	status = cancelled	По cancelled_at DESC

13.1 Два режима чата

Ключевое изменение относительно стандартной модели

Чат открывается сразу после отправки запроса (pending) — ограниченный режим. После принятия — переходит в полный режим. История pre-booking чата сохраняется.

Параметр	Pre-booking чат (pending)	Full чат (accepted)
Когда открывается	Сразу после POST /bookings	После PATCH /bookings/:id/accept
Кто может писать	Пассажир → водителю	Обе стороны
Лимит сообщений	Макс 10 сообщений	Без лимита
Телефон виден?	НЕТ — заменяется на [скрыто]	ДА — полный номер
Фильтр контента	Телефоны, адреса соцсетей → [скрыто]	Только телефон до accepted
Тема сообщений	Только вопросы о поездке	Любые
После completed	Не применимо	Читать можно, писать нельзя. 48ч.
chat_phase в БД	pre_booking	full

13.2 UX — карточка запроса у водителя

Водитель видит запрос в стиле мессенджера: фото пассажира, имя, рейтинг, и сразу под ними — вопросы из pre-booking чата. Не нужно переходить никуда — всё видно в одной карточке. Кнопки "Принять" и "Отклонить" крупные, одним тапом.

```
// Карточка запроса — данные для рендера
{
  booking_id: "uuid",
  passenger: {
    name: "Айгуль М.",
    avatar_url: "...",
    rating: 4.8,
    rating_count: 23,
    is_new: false
  },
  seats_count: 2,
  comment: "Буду с небольшой сумкой",
  expires_at: "2026-04-20T09:00:00Z", // таймер
  pre_chat_messages: [ // ← вопросы видны прямо в карточке
    { text: "Где точно встречаемся?", created_at: "..." },
    { text: "Есть ли место для рюкзака?", created_at: "..." }
  ]
}
```

13.3 Статусы бронирования в реальном времени

Пассажир видит статус в реальном времени через Socket.IO — как в мессенджере, без обновления страницы.

Статус	Что видит пассажир	Socket событие
pending	❑ Запрос отправлен	booking:new_request
viewed	❑ Водитель просматривает	booking:viewed (новый)
accepted	❑ Принято! Чат открыт	booking:accepted
rejected	❑ Отклонено	booking:rejected
expired	❑ Нет ответа (1 час)	booking:expired

Новый статус viewed

Когда водитель открыл карточку запроса — пассажир получает Socket событие booking:viewed. Это снижает тревогу ожидания — пассажир видит что водитель смотрит на запрос. Аналог "галочки прочитано" в WhatsApp.

13.4 Socket.IO события чата

```
// Клиент → Сервер
socket.emit('chat:join',    { booking_id })
socket.emit('chat:send',    { booking_id, text, client_msg_id })
socket.emit('chat:read',    { message_id })
socket.emit('chat:typing',  { booking_id })

// Новое: водитель открыл карточку запроса
socket.emit('booking:view', { booking_id })

// Сервер → Клиент
socket.on('chat:joined',    { booking_id, history, chat_phase })
socket.on('chat:message',   { message })
socket.on('chat:message_sent', { client_msg_id, server_id })
socket.on('chat:read',     { message_id, read_at })
socket.on('chat:typing',    { user_id })
socket.on('chat:phase_changed', { booking_id, new_phase: 'full' }) // при принятии
socket.on('chat:limit_warning', { booking_id, messages_left: 2 }) // pre-booking лимит
socket.on('booking:viewed', { booking_id }) // новое: водитель посмотрел
socket.on('chat:error',     { code, message })
```

13.5 Изменения в таблице messages

Поле	Тип	Описание
chat_phase	VARCHAR(15) DEFAULT 'pre_booking'	pre_booking full
is_filtered	BOOLEAN DEFAULT FALSE	Контент был отфильтрован (номер/адрес заменён)
original_text	TEXT NULL	Оригинал до фильтрации (только для поддержки)

Новое поле в bookings — viewed_at

bookings.viewed_at TIMESTAMPTZ NULL — когда водитель открыл карточку запроса. Используется для Socket события booking:viewed и для аналитики (время реакции водителя).

13.6 Безопасность чата

Правило	Реализация
Аутентификация Socket	JWT в handshake (socket.handshake.auth.token)
Авторизация на chat:join	Сервер: user.id = booking.passenger_id OR trip.driver_id
Pre-booking: только пассажир пишет	Водитель может читать и отвечать, но не инициировать
Rate limit сообщений	20 сообщений в минуту на пользователя
Pre-booking лимит	Макс 10 сообщений. При 8 — предупреждение chat:limit_warning
Фильтр телефонов	regex /\+?[0-9\s\-\]{7,}/ → [номер скрыт] в pre_booking фазе
Фильтр соцсетей	telegram.me, wa.me, instagram.com → [ссылка скрыта] в pre_booking
Макс длина сообщения	2000 символов
Хранение оригинала	original_text хранится для поддержки, пользователю недоступен

14.1 Когда запрашивается оценка

Через 2 часа после `departure_at + estimated_duration_min` — cron job меняет статус поездки на `completed` и создаёт уведомления для обеих сторон с просьбой оценить. Окно для оценки — 7 дней. После этого возможность оценить закрывается.

14.2 Теги оценок

Категория	Теги
Положительные о водителе	Приехал вовремя · Чистая машина · Безопасное вождение · Приятное общение · Комфортная поездка
Отрицательные о водителе	Опоздал · Грязная машина · Опасное вождение · Грубость · Машина не соответствует описанию
Положительные о пассажире	Пришёл вовремя · Вежливый · Нет лишнего багажа · Приятное общение
Отрицательные о пассажире	Опоздал · Много багажа · Грубость · No-show

14.3 Алгоритм расчёта рейтинга

```
// Triggered after insert into ratings table (DB trigger)
function updateUserRating(userId) {
  const recent = await db.ratings.findMany({
    where: { ratee_id: userId },
    orderBy: { created_at: 'desc' },
    take: 50 // только последние 50 оценок
  });
  if (recent.length < 3) {
    // Меньше 3 оценок — рейтинг не отображается, показываем "Новый"
    return;
  }
  const avg = recent.reduce((a, r) => a + r.score, 0) / recent.length;
  await db.users.update({
    where: { id: userId },
    data: {
      rating: Math.round(avg * 100) / 100, // 4.87 округление
      rating_count: recent.length
    }
  });
  // Проверка на автоматические действия
  if (avg < 4.0) notifyUser('rating_warning', userId);
  if (avg < 3.5) notifyAdmin('low_rating_review', userId);
  if (avg < 3.0) suspendDriver(userId);
}
```

14.4 Защита от накрутки

Правило	Детали
Одна оценка на одну поездку на одного участника	UNIQUE(trip_id, rater_id, ratee_id)
Нельзя оценить без completed поездки	Только если booking.status = completed
Нельзя оценить после 7 дней	Проверка NOW() - trip.departure_at < 7 days
Нельзя менять оценку после отправки	PATCH на ratings не разрешён
Фильтр одинаковых отзывов от одного автора	Если 5+ отзывов одного пользователя с одинаковым текстом — алерт администратору

15.1 Каналы

Канал	MVP?	Когда используется
Telegram Bot	Да	Основной канал в MVP — даже если юзер не в Mini App, получит сообщение от бота
Push (FCM)	Нет (Этап 2)	Когда выйдет Flutter приложение
In-app	Да	Иконка колокольчика со счётчиком
Email	Нет (Этап 2)	Для администраторов
SMS	Только OTP	Слишком дорого для обычных уведомлений

15.2 Полный каталог уведомлений

Тип	Получатель	Триггер	Текст (ru)
new_booking_request	Водитель	Пассажир создал booking	Новый запрос от {name}. {from}→{to}, {seats} места.
booking_accepted	Пассажир	Водитель принял	{driver_name} принял ваш запрос! Чат открыт.
booking_rejected	Пассажир	Водитель отклонил	К сожалению, водитель отклонил запрос.
booking_expired	Пассажир	Cron: прошёл 1 час без ответа	Водитель не ответил. Попробуйте другую поездку.
booking_cancelled_by_passenger	Водитель	Пассажир отменил	{name} отменил бронирование.
trip_cancelled	Все пассажиры	Водитель отменил поездку	Поездка {from}→{to} на {date} отменена.
trip_reminder	Все участники	Cron: за 2 часа до departure_at	Напоминание: поездка через 2 часа.
trip_completed_rate	Все участники	Cron: через 2 часа после estimated прибытия	Оцените поездку {from}→{to}.
new_message	Собеседник	chat:send	{sender_name}: {text_preview}
rating_received	Оценённый	Insert в ratings	{rater_name} поставил вам оценку {score}.
rating_warning	Водитель	rating < 4.0	Ваш рейтинг снизился до {rating}. Улучшите его.
verification_approved	Водитель	Админ verify	Поздравляем! Вы верифицированы как водитель.
verification_rejected	Водитель	Админ reject	Верификация отклонена. Причина: {reason}.
verification_need_docs	Водитель	Админ request_more	Нужны дополнительные документы: {list}.
account_blocked	Пользователь	Админ block	Ваш аккаунт заблокирован. Причина: {reason}.

Тип	Получатель	Триггер	Текст (ru)
new_complaint	Администратор	Insert в complaints	Новая жалоба в категории {category}.

16.1 Политика отмен (MVP)

Сценарий	Действие системы	Последствия
Водитель отменяет поездку	Все bookings → cancelled_by_driver, уведомления всем	cancellations_30d++. Если >5 — блок публикации на 7 дней.
Пассажир отменяет >24 до поездки	booking.status = cancelled_by_passenger, seats_available возвращается	Нет штрафа
Пассажир отменяет <24 до поездки	booking.status = cancelled_late, seats_available возвращается	Пометка в истории. Этап 3 — штраф.
Пассажир не явился (no-show)	Водитель нажимает "Не явился" после времени отправления	booking.status = no_show, рейтинг пассажира −0.5
Водитель не явился	Пассажир нажимает "Водитель не приехал"	Автоматическая жалоба, админ разбирает

16.2 Флоу подачи жалобы

1 Пользователь нажимает "Пожаловаться" на профиле или поездке

Кнопка в Mini App/Flutter в трёх точках

2 Выбор категории

Безопасность / Мошенничество / Грубость / No-show / Другое

3 Текст жалобы (до 2000 символов)

Обязательно

4 Опционально — скриншоты

До 5 файлов

5 POST /complaints

Создание записи со статусом new

6 Уведомление администратору

Socket + email (Этап 2)

7 Администратор разбирает

Смотрит переписку в чате, историю поездок, предыдущие жалобы

8 Решение

resolved (с комментарием) / dismissed (необоснованная)

9 Уведомление заявителю

С решением

1 Возможные санкции

0 Предупреждение / Suspend на N дней / Постоянная блокировка

16.3 Категории жалоб и приоритеты

Категория	Приоритет	SLA ответа
Безопасность (угроза жизни/здоровью)	P0 — критичный	До 1 часа

Категория	Приоритет	SLA ответа
Мошенничество	P1 — высокий	До 6 часов
No-show, грубость	P2 — средний	До 24 часов
Другое	P3 — низкий	До 72 часов

17.1 Разделы и приоритеты

Раздел	Назначение	MVP?
Dashboard (главная)	KPI карточки + ключевые графики	Да
Верификация водителей	Очередь pending с фильтрами	Да
Пользователи	Список всех users с поиском, профиль, история, блокировка	Да
Поездки	Список всех trips с фильтрами, детали, возможность принудительной отмены	Да
Жалобы	Очередь с приоритетами, разбор, санкции	Да
Города	Управление справочником cities (только суперадмин)	Да
Администраторы	CRUD администраторов (только суперадмин)	Да
Аудит-логи	Действия администраторов	Да
Рассылки	Массовые уведомления	Нет (Этап 2)
Финансы	Отчёты о комиссиях	Нет (Этап 3)

17.2 Раздел "Верификация" — детально

Элемент UI	Функция
Список pending заявок	Фильтры: дата подачи, имя, госномер. Сортировка: по дате ASC (FIFO)
Карточка заявки	Все данные авто + галерея 4 фото в нормальном размере + возможность открыть fullscreen
Просмотр фото	Zoom + rotate + яркость (для проверки на подлинность)
История заявок водителя	Если уже подавал ранее — показываем историю
Кнопка "Одобрить"	Меняет status на verified, добавляет driver в user.roles
Кнопка "Отклонить"	Требуется указать причину (обязательно)
Кнопка "Запросить документы"	Чеклист: какие документы нужно переподать
Таймер SLA	Красная рамка если прошло >24 часа с подачи

17.3 Раздел "Пользователи"

Колонка в списке	Сортировка/фильтр
Аватар + имя	—
Телефон	Поиск
Роли	Фильтр (passenger/driver/оба)
Рейтинг	Сортировка ASC/DESC
Количество поездок	Сортировка
Дата регистрации	Сортировка ASC/DESC

Колонка в списке	Сортировка/фильтр
Статус блокировки	Фильтр (активен/заблокирован)
Количество жалоб	Сортировка DESC

18.1 KPI-карточки (верх дашборда)

Карточка	Формула	Период
Всего пользователей	COUNT(users WHERE deleted_at IS NULL)	Всего / +за 7 дней / +за 30 дней
Активные водители	COUNT(DISTINCT driver_id FROM trips WHERE created_at > NOW() - 7d)	За 7 дней
Опубликованных поездок	COUNT(trips WHERE status = active)	Сейчас
Завершённых поездок	COUNT(trips WHERE status = completed)	За 7 дней / 30 дней
Процент принятия	accepted / (accepted + rejected) × 100	За 7 дней
В очереди верификации	COUNT(driver_profiles WHERE status = pending)	Сейчас
Открытых жалоб	COUNT(complaints WHERE status IN (new, in_review))	Сейчас
Средний рейтинг водителей	AVG(users.rating WHERE "driver" IN roles AND rating_count >= 3)	Сейчас

18.2 Графики

График	Библиотека	Данные	Период
Регистрации по дням	Recharts LineChart	COUNT(users) GROUP BY DATE(created_at)	30 дней
Поездки по дням	Recharts BarChart (stacked)	COUNT GROUP BY DATE, status	30 дней
Топ-10 маршрутов	Recharts BarChart horizontal	COUNT GROUP BY (origin_city, destination_city)	30 дней
Активность по часам	Recharts Heatmap	COUNT GROUP BY day_of_week, hour	7 дней
Средний рейтинг по дням	Recharts LineChart	AVG(score) GROUP BY DATE	30 дней
Воронка регистрации	Recharts FunnelChart	См. 18.3	За период

18.3 Воронка онбординга

Этап	Событие в коде	Описание
1. Открыли Mini App/App	app_opened event	Трекинг на клиенте
2. Начали регистрацию	users.created_at	Создали запись в users
3. Подтвердили телефон	users.phone_verified_at	—
4. Заполнили имя/аватар	users.name IS NOT NULL	—
5. Первое действие	Первый search/trip/booking	Активация

Этап	Событие в коде	Описание
6. Вторая сессия	app_opened через 1+ день	Retention Day-1
7. Вторая поездка	Второй booking со status = completed	Core retention

18.4 Экспорт

Все графики и таблицы имеют кнопку "Экспорт в CSV/Excel". Администратор может выгрузить данные для отчётности. Суперадмин может экспортировать полную выгрузку пользователей (без чувствительных данных типа фото документов).

19.1 Соглашения

Правило	Детали
Base URL	https://api.popytchik.kg/v1
Формат	JSON с Content-Type: application/json; charset=utf-8
Авторизация	Authorization: Bearer
Версионирование	Через URL /v1/. Breaking changes → /v2/.
Пагинация	Cursor-based: ?cursor=xxx&limit;=20. Возврат: {data, next_cursor}
Ошибки	HTTP status + {error: {code, message, details}}. Коды в формате UPPER_SNAKE.
Даты	ISO 8601 в UTC (2026-04-17T12:00:00Z)
Идемпотентность	Header Idempotency-Key (UUID) для POST /trips, POST /bookings
Rate limit headers	X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset

19.2 Полный список эндпоинтов

Auth

Метод	Путь	Описание	Auth
POST	/auth/telegram	Вход через Telegram WebApp	Нет
POST	/auth/phone/send-otp	Отправить OTP	Нет
POST	/auth/phone/verify	Подтвердить OTP	Нет
POST	/auth/refresh	Обновить access token	Refresh
POST	/auth/logout	Выйти (revoke refresh)	JWT
POST	/auth/admin/login	Вход админа email + пароль + 2FA	Нет

Users

Метод	Путь	Описание	Auth
GET	/users/me	Свой профиль	JWT
PATCH	/users/me	Обновить имя/аватар/язык	JWT
DELETE	/users/me	Удалить аккаунт (soft delete)	JWT
GET	/users/:id	Публичный профиль (для карточки)	JWT
GET	/users/:id/ratings	Отзывы о пользователе	JWT
POST	/users/avatar	Загрузить аватар	JWT

Drivers

Метод	Путь	Описание	Auth
POST	/drivers/verification	Подать заявку на верификацию	JWT
GET	/drivers/verification/status	Статус заявки	JWT
POST	/drivers/verification/upload	Загрузить документ	JWT
GET	/drivers/me/stats	Своя статистика водителя	JWT + driver

Trips

Метод	Путь	Описание	Auth
POST	/trips	Создать поездку (idempotency-key)	JWT + driver
GET	/trips	Поиск поездок	JWT
GET	/trips/:id	Детали поездки	JWT
PATCH	/trips/:id	Обновить поездку (только автор)	JWT + driver
DELETE	/trips/:id	Отменить поездку	JWT + driver
GET	/trips/my	Мои поездки (водитель)	JWT + driver

Bookings

Метод	Путь	Описание	Auth
POST	/bookings	Создать бронирование	JWT
GET	/bookings/my	Мои бронирования (все роли)	JWT
GET	/bookings/incoming	Входящие запросы (водитель)	JWT + driver
GET	/bookings/:id	Детали брони	JWT
PATCH	/bookings/:id/accept	Принять (только водитель)	JWT + driver
PATCH	/bookings/:id/reject	Отклонить (только водитель)	JWT + driver
PATCH	/bookings/:id/cancel	Отменить (обе стороны)	JWT
PATCH	/bookings/:id/no-show	Отметить не явился (водитель)	JWT + driver

Chat (REST — история; WebSocket — realtime)

Метод	Путь	Описание	Auth
GET	/chats/:booking_id/messages	История чата (пагинация)	JWT
POST	/chats/:booking_id/messages	Отправить (fallback без WS)	JWT
PATCH	/messages/:id/read	Отметить прочитанным	JWT

Ratings

Метод	Путь	Описание	Auth
POST	/ratings	Оставить оценку	JWT
GET	/ratings/pending	Поездки ждущие оценки	JWT

Complaints

Метод	Путь	Описание	Auth
POST	/complaints	Подать жалобу	JWT
GET	/complaints/my	Мои жалобы	JWT

Utility

Метод	Путь	Описание	Auth
GET	/cities	Список активных городов КГ	Нет
GET	/routes/price-suggestion	Средняя цена на маршруте	JWT
GET	/notifications	Мои уведомления	JWT
PATCH	/notifications/:id/read	Отметить прочитанным	JWT
GET	/health	Health check	Нет

Admin API (/admin/*)

Метод	Путь	Описание	Auth
GET	/admin/verifications	Очередь верификации	admin
GET	/admin/verifications/:id	Детали заявки	admin
PATCH	/admin/verifications/:id/approve	Одобрить	admin
PATCH	/admin/verifications/:id/reject	Отклонить (требуется reason)	admin
PATCH	/admin/verifications/:id/request-docs	Запросить документы	admin
GET	/admin/users	Список пользователей	admin
GET	/admin/users/:id	Детали пользователя	admin
PATCH	/admin/users/:id/block	Заблокировать	admin
PATCH	/admin/users/:id/unblock	Разблокировать	admin
GET	/admin/trips	Список поездок	admin
PATCH	/admin/trips/:id/cancel	Принудительная отмена	admin
GET	/admin/complaints	Список жалоб	admin
PATCH	/admin/complaints/:id/resolve	Разрешить жалобу	admin
GET	/admin/cities	Список городов	admin
POST	/admin/cities	Добавить город	superadmin
PATCH	/admin/cities/:id	Обновить город	superadmin
GET	/admin/admins	Список администраторов	superadmin
POST	/admin/admins	Создать администратора	superadmin
GET	/admin/audit-log	Аудит действий	admin (свои), superadmin (все)
GET	/admin/analytics/kpi	KPI метрики	admin
GET	/admin/analytics/charts/:name	Данные для графика	admin

20.1 Подключение

```
// Клиент
const socket = io('wss://api.popytchik.kg', {
  auth: { token: accessToken },
  transports: ['websocket'], // без polling для мобильных
  reconnection: true,
  reconnectionDelay: 1000,
  reconnectionAttempts: 5
});

socket.on('connect', () => console.log('connected'));
socket.on('disconnect', reason => console.log('disconnected', reason));
socket.on('auth:error', () => refreshTokenAndReconnect());
```

20.2 Полный каталог событий

Событие	Направление	Когда	Данные
connect	S→C	После успешной аутентификации	—
auth:error	S→C	Token недействителен	{code}
chat:join	C→S	Подключиться к чату	{booking_id}
chat:joined	S→C	Ответ на join	{booking_id, history}
chat:send	C→S	Отправить сообщение	{booking_id, text, client_msg_id}
chat:message	S→C	Новое сообщение в чате	{message}
chat:message_sent	S→C	АСК отправки (идемпотентность)	{client_msg_id, server_id}
chat:read	C→S / S→C	Отметка прочтения	{message_id}
chat:typing	C→S / S→C	Индикатор печати	{booking_id, user_id}
booking:new_request	S→C (driver)	Новый booking на поездку	{booking, trip}
booking:accepted	S→C (passenger)	Бронирование принято	{booking}
booking:rejected	S→C (passenger)	Отклонено	{booking}
booking:cancelled	S→C (other party)	Другая сторона отменила	{booking, cancelled_by}
trip:cancelled	S→C (passengers)	Водитель отменил поездку	{trip}
notification:new	S→C	Общее уведомление	{notification}

20.3 Горизонтальное масштабирование WebSocket

В MVP — один Socket.IO инстанс, подключения хранятся в памяти. При масштабировании (Этап 2) — Redis Adapter для Socket.IO, который синхронизирует события между инстансами. Nginx sticky sessions через ip_hash или cookie.

Задача	Расписание	Что делает
expire_bookings	* * * * * (каждую минуту)	UPDATE bookings SET status=expired WHERE status=pending AND expires_at<NOW()
auto_complete_trips	* /15 * * * *	Поездки где departure + duration + 2h < NOW() → status = completed, создание rating request
trip_reminder_2h	* /5 * * * *	Поездки через 2 часа — уведомление всем участникам
rating_request	* /15 * * * *	После auto_complete — отправка уведомления "оцените поездку"
recalc_cancellations_30d	0 3 * * *	Пересчёт driver_profiles.cancellations_30d для всех
cleanup_expired_otp	0 * /6 * * *	DELETE FROM otp_codes WHERE expires_at < NOW() - 1 day
cleanup_expired_refresh	0 4 * * *	DELETE FROM refresh_tokens WHERE expires_at < NOW() - 1 day
db_backup	0 3 * * *	pg_dump → /backups/. Удаление старше 30 дней.
files_backup	0 4 * * *	rsync файлов на бэкап-сервер
analytics_recalc	* /30 * * * *	Инкрементальный пересчёт KPI для дашборда (материализованные вьюхи)
escalate_verifications	0 * /4 * * *	Заявки pending > 24h → уведомление суперадмину
escalate_complaints	* /30 * * * *	Жалобы P0 без ответа > 1h — SMS суперадмину

21.1 Реализация

В MVP — node-cron внутри основного API процесса. Для предотвращения двойного выполнения при нескольких инстансах — перед каждой задачей INSERT в таблицу cron_locks с UNIQUE ограничением. На Этапе 2 — BullMQ + Redis для распределённых очередей.

22.1 Стратегия для слабого 4G в регионах КГ

Уровень	Решение
API retry	Axios interceptor: 3 попытки с exponential backoff (1s, 2s, 4s) при 5xx и network error
Показать пользователю	Toast "Нет связи. Проверьте интернет." Кнопка "Повторить"
Offline queue (Flutter)	Hive: очередь действий, выполняется при восстановлении связи
Offline queue (Mini App)	localStorage: критичные действия (отправка сообщения в чат)
Cache (Flutter)	dio_cache_interceptor: GET запросы кэшируются на 5 минут
Cache (Mini App)	TanStack Query с staleTime 5 минут
Optimistic UI	Отправка сообщения в чат, создание booking — UI сразу показывает, потом корректирует
Idempotency	POST /trips и POST /bookings идемпотентны — повторный вызов не создаёт дубль
WebSocket reconnect	Автоматический переподключение, восстановление chat:join комнат

22.2 Формат ошибок API

```
HTTP 400+
Content-Type: application/json
{
  "error": {
    "code": "SEATS_NOT_AVAILABLE",
    "message": "Свободных мест больше нет",
    "message_ky": "Бош орундар жок",
    "details": {
      "trip_id": "uuid",
      "seats_requested": 2,
      "seats_available": 0
    },
    "request_id": "uuid-для-саппорта"
  }
}
```

Клиент показывает пользователю message на выбранном языке, логирует request_id для обращения в поддержку.

22.3 Каталог кодов ошибок

Код	HTTP	Когда
VALIDATION_ERROR	400	Неверный формат запроса
UNAUTHORIZED	401	Токен недействителен или отсутствует
TOKEN_EXPIRED	401	Access token истёк, надо обновить
FORBIDDEN	403	Нет прав на действие

Код	HTTP	Когда
NOT_FOUND	404	Ресурс не существует
CONFLICT	409	Конфликт состояния (мест нет, дубль)
SEATS_NOT_AVAILABLE	409	Мест больше нет
BOOKING_ALREADY_EXISTS	409	Пассажир уже имеет активный booking на поездку
TRIP_NOT_ACTIVE	409	Нельзя бронировать неактивную поездку
RATE_LIMITED	429	Превышен лимит запросов
OTP_EXPIRED	410	OTP истёк
OTP_WRONG	400	Неверный OTP
OTP_TOO_MANY_ATTEMPTS	429	Превышены попытки ввода OTP
INTERNAL_ERROR	500	Внутренняя ошибка сервера
SERVICE_UNAVAILABLE	503	Сервис временно недоступен

23.1 Процесс перевода

Этап	Ответственный	Результат
Разработчик пишет ключ на русском	Developer	locales/ru.json
Накопление ключей за спринт	Developer	PR с изменениями ru.json
Перевод на кыргызский	Переводчик (native speaker)	locales/ky.json
Review перевода	Product Owner + носитель	Утверждённые переводы
Merge в main	Developer	Обновление в staging
Тестирование на реальном устройстве	QA	Проверка переноса строк, обрезания

23.2 Стек локализации

Клиент	Библиотека	Формат файлов
Telegram Mini App (React)	i18next + react-i18next	locales/ru.json, locales/ky.json
Flutter	easy_localization	assets/translations/ru.json, ky.json
Next.js Web	next-intl	messages/ru.json, ky.json
Admin Panel	next-intl	Только русский в MVP
Backend (сообщения об ошибках)	accept-language header	src/locales/*.json
Telegram Bot	i18next-node	Читает users.language при отправке

23.3 Правила именования ключей

```
// Структура ключей по модулям
{
  "common": {
    "buttons": {
      "submit": "Отправить",
      "cancel": "Отменить"
    }
  },
  "auth": {
    "phone": {
      "enter": "Введите номер телефона",
      "invalid": "Неверный формат номера"
    }
  },
  "trips": {
    "create": {
      "title": "Создать поездку",
      "from_placeholder": "Откуда едете?"
    }
  }
}
```

```
},  
"errors": {  
  "SEATS_NOT_AVAILABLE": "Свободных мест больше нет"  
}  
}
```

Правила:

- Глубина не более 3 уровней
- Ключи в snake_case
- Одинаковая структура в ru.json и ky.json
- Запрет на склеивание строк в коде — только через интерполяцию

24.1 Правовая база

Закон КР «О персональных данных» № 58 от 14.04.2008. Платформа хранит персональные данные (ФИО, телефон, фото документов) — нужно согласие пользователя при регистрации и возможность удалить данные по запросу.

24.2 Согласие при регистрации

Чекбокс при первом входе: "Я согласен с Политикой конфиденциальности и Пользовательским соглашением". Дата согласия сохраняется в `users.terms_accepted_at`. Без согласия регистрация невозможна.

24.3 Удаление аккаунта

Действие	Что происходит с данными
Пользователь нажимает "Удалить аккаунт"	Подтверждение через OTP
<code>DELETE /users/me</code>	Soft delete: <code>users.deleted_at = NOW()</code>
<code>users.name</code>	Заменяется на "Удалённый пользователь"
<code>users.phone, telegram_id, avatar</code>	Обнуляются (NULL)
Фото документов водителя	Физически удаляются с диска через 30 дней (cron)
Сообщения в чатах	Остаются (важны для других участников), отправитель = "Удалённый"
История поездок	Остаётся для статистики, обезличена
Оценки и отзывы	Остаются, автор = "Удалённый пользователь"
Через 30 дней — hard delete	Полное физическое удаление записи <code>users</code> и <code>driver_profiles</code>

24.4 Экспорт своих данных (право на портатбельность)

Эндпоинт `GET /users/me/export` возвращает JSON со всеми данными пользователя: профиль, поездки, бронирования, чаты, оценки. Файл доступен для скачивания 24 часа. В MVP — запрос обрабатывается вручную администратором за 72 часа, на Этапе 2 — автоматизирован.

25.1 Архитектура бота

Telegram Bot это отдельный воркер-процесс Node.js. Слушает обновления от Telegram через Long Polling (в MVP) или Webhook (Этап 2). Имеет доступ к той же БД что и API. Mini App запускается через кнопку в боте (inline button с web_app URL).

25.2 Команды бота

Команда	Описание
/start	Приветствие + кнопка "Открыть приложение" (запускает Mini App)
/help	Справка: что умеет платформа
/mytrips	Быстрый список моих активных поездок/бронирований
/support	Контакт поддержки: @popytchik_support
/lang	Переключить язык (ru ↔ ky)
/unsubscribe	Отписаться от уведомлений (сохраняем в users.notifications_enabled=false)
/subscribe	Подписаться обратно

25.3 Формат уведомлений

```
// Пример формата для new_booking_request
□ <b>Новый запрос на поездку</b>
От: <b>Асан К.</b> (рейтинг □ 4.8)
Поездка: Бишкек → Ош
Дата: 20 апреля, 08:00
Мест: 2

[ Открыть заявку ] <-- кнопка, ведёт в Mini App на конкретный экран

Правила форматирования:
- HTML mode (parse_mode: 'HTML')
- Эмодзи в начале для визуального разделения
- Кнопки deep-link в Mini App с query параметрами
- Длина сообщения не более 1000 символов
```

26.1 Назначение и ключевые отличия

Web клиент решает две задачи: SEO-сайт для привлечения через поисковики и полноценный кабинет для пользователей без Telegram. Единственный клиент где SEO критично — страницы поездок рендерятся через Next.js SSR/ISR.

Особенность	Web (отличие от Mini App / Flutter)
Авторизация	Phone+Password, Google, Apple. Нет Telegram initData в браузере.
Refresh token	HttpOnly + Secure + SameSite=Strict cookie (не sessionStorage).
SEO	SSR/ISR через Next.js. Google индексирует маршруты и профили водителей.
Загрузка файлов	File input + canvas API для сжатия. Нет нативной камеры.
Уведомления	WebSocket (in-app) + Telegram Bot. Web Push (FCM) — Этап 2.
Offline	Нет offline режима. Axios retry (3 попытки, exponential backoff).
Layout desktop	Split view: фильтры список детали. Три колонки > 1024px.
Layout mobile	Одна колонка < 768px. Bottom sheet для деталей поездки.
Видимость телефона	Те же правила — скрыт до booking.status = accepted.

26.2 Структура страниц

Страница	URL	SSR/CSR	Auth	Описание
Главная	/	SSR	Нет	Лендинг: hero с поиском, популярные маршруты, СТА
Поиск поездок	/trips	SSR+CSR	Нет	SSR первые 20 результатов, CSR фильтрация. Без auth — просмотр.
Детали поездки	/trips/:id	SSR	Нет	Полные данные. SEO meta. Кнопка "Забронировать" → Deferred Action.
Бронирование	/trips/:id/book	CSR	Да	Форма: кол-во мест, комментарий. Deferred Action если не авторизован.
Мои поездки	/my/trips	CSR	Да	История водителя и пассажира. Вкладки: активные, прошедшие.
Мои бронирования	/my/bookings	CSR	Да	Входящие (водитель) + мои запросы (пассажир). Принять/отклонить.
Чат	/my/bookings/:id/chat	CSR	Да	Socket.IO чат. Двухпанельный layout. Tab title notification.
Профиль	/profile	CSR	Да	Имя, аватар, язык, пароль, смена номера.
Верификация водителя	/profile/driver	CSR	Да	5-шаговый wizard. File input + canvas compress.
Уведомления	/notifications	CSR	Да	Список уведомлений, cursor-based пагинация.
Оценить поездку	/trips/:id/rate	CSR	Да	Форма оценки 1-5 + теги + комментарий.
Жалоба	/complaint	CSR	Да	Форма жалобы. Категория + текст + скриншоты.

Страница	URL	SSR/CSR	Auth	Описание
Публикация поездки	/trips/create	CSR	Да (driver)	5-шаговый wizard. Прогресс в localStorage.
Вход	/auth/login	CSR	Нет	Phone+Password, Google, Apple. Deferred Action после входа.
Регистрация	/auth/register	CSR	Нет	Phone → OTP → профиль → пароль.
Профиль водителя	/drivers/:id	SSR	Нет	Публичный профиль. SEO. Рейтинг, отзывы, поездки.
Удалить аккаунт	/profile/delete	CSR	Да	Подтверждение OTP → soft delete.
Экспорт данных	/profile/export	CSR	Да	GET /users/me/export → JSON скачивание.

26.3 Поиск поездок — Web специфика

Поисковая форма — главный элемент главной страницы. Стиль Google Flights: крупный hero блок с четырьмя полями (откуда, куда, дата, мест). На desktop — split view: слева фильтры, в центре список, справа детали. Клик на поездку — детали открываются справа без перехода на новую страницу.

UX паттерн	Источник	Реализация в Koshu
Hero поисковая форма	Google Flights	Input C (card field) + Amber кнопка на главной
URL-based фильтры	Google Flights	?from=bishkek&to=osh&date=2026-05-01 — SEO friendly, шаринг ссылок
SSR первого экрана	Google Flights	getServerSideProps загружает первые 20 поездок — мгновенный FCP
Split view desktop	Airbnb / Booking	Список слева + детали справа без перезагрузки страницы
Популярные маршруты	BlaBlaCar	6 карточек на главной: Бишкек→Ош, Бишкек→Каракол и др.
Timeline поездок по дням	BlaBlaCar	Вкладка "На этой неделе" — поездки по дням
Бесконечная подгрузка	Kayak	IntersectionObserver + TanStack Query cursor-based
Активные фильтры	BlaBlaCar	Иконка фильтра меняет цвет когда фильтр применён
Карта маршрута	Booking	Leaflet + OpenStreetMap — точка отправления на карте

26.4 Breakpoints — адаптивность

Название	Ширина	Layout	Навигация	Поиск
Mobile	< 768px	1 колонка	Bottom tab bar (4 вкладки)	Fullscreen форма → список
Tablet	768-1024px	2 колонки	Sidebar слева (свёрнутый)	Фильтры + список
Desktop	> 1024px	3 колонки	Top navbar + sidebar	Фильтры список детали (split view)

26.5 Чат на Web

Socket.IO — те же события что Mini App и Flutter. Desktop: двухпанельный layout — слева список активных бронирований, справа чат. Mobile: fullscreen чат с кнопкой назад.

```
// Web-специфика — уведомление через title браузерной вкладки
```

```
document.addEventListener("visibilitychange", () => {
  if (document.hidden && unreadCount > 0) {
    document.title = `(${unreadCount}) Kosho – новые сообщения`;
  } else {
    document.title = "Kosho";
    markAllAsRead();
  }
});
// Те же Socket.IO события что Mini App
const socket = io("wss://api.kosho.kg", {
  auth: { token: getAccessToken() },
  transports: ["websocket"],
});
socket.emit("chat:join", { booking_id });
socket.on("chat:message", (msg) => appendMessage(msg));
```

26.6 Верификация водителя на Web

Отличие от Flutter

Flutter использует image_picker + нативную камеру. Web — file input с accept="image/*" + сжатие через canvas API. Требования одинаковые: min 800×600px, max 5MB, JPEG/PNG.

```
async function compressImage(file: File, maxMB = 5): Promise<Blob> {
  const img = await createImageBitmap(file);
  const canvas = document.createElement("canvas");
  const scale = Math.min(1, Math.min(1600 / img.width, 1200 / img.height));
  canvas.width = Math.max(800, img.width * scale);
  canvas.height = Math.max(600, img.height * scale);
  canvas.getContext("2d")!.drawImage(img, 0, 0, canvas.width, canvas.height);
  let quality = 0.9;
  let blob = await new Promise<Blob>(r => canvas.toBlob(b => r(b!), "image/jpeg", quality));
  while (blob.size > maxMB * 1024 * 1024 && quality > 0.5) {
    quality -= 0.1;
    blob = await new Promise<Blob>(r => canvas.toBlob(b => r(b!), "image/jpeg", quality));
  }
  return blob;
}
```

26.7 Публикация поездки — Web wizard

Шаг	URL	Сохранение прогресса
1 — Маршрут	/trips/create/route	localStorage: kosho_trip_draft
2 — Дата и время	/trips/create/datetime	localStorage: обновление
3 — Места и цена	/trips/create/seats	localStorage: обновление
4 — Дополнительно	/trips/create/extras	localStorage: обновление
5 — Пред просмотр	/trips/create/preview	При публикации — localStorage очищается

Защита от потери данных

Если пользователь случайно закрыл вкладку или обновил страницу — черновик сохранён в localStorage. При следующем открытии /trips/create показывается toast: "У вас есть незавершённая поездка. Продолжить?"

26.8 SEO — детальная спецификация

Страница	Title	Description	Canonical
/	Kosho — попутчики КГ	Найди попутчика Бишкек-Ош, Каракол, Нарын	kosho.kg
/trips?from=bishkek&to=osh	Поездки Бишкек → Ош Kosho	X поездок на {date}. Цены от Y сом.	kosho.kg/trips?from=...
/trips/:id	{from} → {to} · {date}	Поездка {from}→{to} {date}. {seats} мест, {price} сом.	kosho.kg/trips/:id
/drivers/:id	{name} — водитель Kosho	{name}, рейтинг {rating}★, {trips} поездок	kosho.kg/drivers/:id

```
// Next.js App Router — динамический generateMetadata
export async function generateMetadata({ params }): Promise<Metadata> {
  const trip = await getTrip(params.id);
  return {
    title: `${trip.origin_city} → ${trip.destination_city} · ${formatDate(trip.departure_at)} | Kosho`,
    description: `Поездка ${trip.origin_city}→${trip.destination_city}. `
      + `${trip.seats_available} мест, от ${trip.price_per_seat} сом. `
      + `Водитель ${trip.driver.name} (${trip.driver.rating})`,
    openGraph: {
      title: `${trip.origin_city} → ${trip.destination_city}`,
      images: [{ url: `https://kosho.kg/api/og/trip/${params.id}` }],
    },
    alternates: { canonical: `https://kosho.kg/trips/${params.id}` },
  };
}
```

26.9 Sitemap и robots.txt

```
// sitemap.ts — Next.js автоматический sitemap
export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const cities = await getCities();
  const routes = generateCityPairs(cities); // все пары активных городов
  const staticPages = ["/", "/auth/login", "/auth/register"];
  const routePages = routes.map(r => ({
    url: `https://kosho.kg/trips?from=${r.from}&to=${r.to}`,
    changeFrequency: "hourly" as const,
    priority: 0.8,
  }));
  return [...staticPages.map(p => ({ url: `https://kosho.kg${p}` })), ...routePages];
}

// robots.txt
User-agent: *
Allow: /
Allow: /trips
```



```

Allow: /trips/*
Allow: /drivers/*
Disallow: /my/*
Disallow: /profile/*
Disallow: /auth/*
Disallow: /complaint
Sitemap: https://kosho.kg/sitemap.xml

```

26.10 Стек Web клиента

Слой	Технология	Версия	Назначение
Framework	Next.js (App Router)	14	SSR/ISR для SEO, routing, API routes
Язык	TypeScript	5.4+	Строгая типизация (strict: true)
Стили	Tailwind CSS	3.4+	Utility-first, консистентность с Mini App
Компоненты	shadcn/ui	latest	Базовые UI компоненты
State	Zustand	4.x	Глобальный auth state, уведомления
Server state	TanStack Query	5.x	Кэш, cursor пагинация, prefetch
HTTP	axios	1.x	Interceptors для auth + retry логики
WebSocket	socket.io-client	4.7+	Чат + real-time уведомления
Формы	react-hook-form + Zod	7.x + 3.x	Валидация форм
Локализация	next-intl	3.x	ru/ky, SSR-friendly
Карты	react-leaflet	4.x	OpenStreetMap, без Google Maps
SEO	Next.js Metadata API	built-in	generateMetadata + sitemap.ts
Иконки	lucide-react	latest	Те же что Mini App и Admin
Шрифт	Nunito (Google Fonts)	—	next/font/google — self-hosted автоматически

26.11 Что НЕ входит в Web MVP

Функция	Этап	Причина
Web Push уведомления (FCM)	Этап 2	Требует Service Worker + FCM setup
Нативная камера через WebRTC	Этап 2	Сложность, file input достаточно для MVP
Offline режим / Service Worker	Этап 2	Нет критической необходимости на Web
Dark mode	Этап 2	Добавляется через Tailwind dark: классы
Карта маршрута с треком	Этап 2	Нужен PostGIS и координаты остановок
Сравнение поездок side-by-side	Этап 2	Кауак паттерн — после MVP
Мультиязычный URL (/ky/trips)	Этап 2	next-intl routing — отдельный спринт

27.1 Принципы

Принцип	Описание
Консистентность	Одни и те же компоненты, цвета и отступы на всех клиентах — Web, Mini App, Flutter
Доступность (a11y)	Контраст текста минимум 4.5:1 (WCAG AA). ARIA labels на иконках. Focus visible.
Локальность	Кириллица Nunito читается одинаково хорошо на русском и кыргызском
Минимализм	Белый фон, мало цвета. Teal только на СТА и акцентах. Amber только на главных действиях.
Мобильность	Все компоненты touch-friendly: минимальная tap target — 44×44px

27.2 Цветовая палитра

Primary — Teal

Токен	HEX	Использование
teal-50	#F0FDFA	Фон badge верифицирован, hover фон
teal-100	#CCFBF1	Фон аватара по умолчанию, background info
teal-400	#2DD4BF	Hover эффекты, border focus
teal-500	#14B8A6	Border focus, иконки
teal-600	#0D9488	Primary кнопки, ссылки, навигация активная — ОСНОВНОЙ
teal-700	#0F766E	Hover primary кнопки, цена в карточке
teal-900	#0D3D38	Заголовки dark, footer фон

Accent — Amber

Токен	HEX	Использование
amber-50	#FFFBEF	Фон уведомлений reminder, pending фон
amber-100	#FEF3C7	Фон badge pending, аватар запасной
amber-400	#FBBF24	Звёзды рейтинга
amber-500	#F59E0B	Submit кнопка "Найти", "Забронировать" — ГЛАВНОЕ ДЕЙСТВИЕ
amber-600	#D97706	Hover amber кнопки

Semantic + Neutrals

Токен	HEX	Использование
success	#10B981	Верифицирован, принято, success state
error	#EF4444	Ошибка, отклонено, заблокирован
warning	#F59E0B	Pending, предупреждение (тот же amber-500)
gray-900	#111827	Основной текст
gray-700	#374151	Вторичный текст

Токен	HEX	Использование
gray-500	#6B7280	Placeholder, muted, caption
gray-300	#D1D5DB	Borders, dividers
gray-100	#F3F4F6	Фон карточек, disabled
gray-50	#F9FAFB	Фон страницы
white	#FFFFFF	Фон карточек, surface

27.3 Типографика — Nunito

Почему Nunito

Полная поддержка кириллицы (в отличие от Plus Jakarta Sans который не поддерживает). Мягкий округлый характер — тёплый и дружелюбный, подходит для продукта доверия. Бесплатный коммерческий, Google Fonts, self-hosted через next/font автоматически.

Токен	Размер	Вес	Использование
display	32px	800 ExtraBold	Hero заголовки, название маршрута на детальной
h1	24px	700 Bold	Заголовки экранов, страниц
h2	18px	700 Bold	Секции, подзаголовки
body-lg	15px	400 Regular	Основной текст, описания
body	14px	600 SemiBold	Текст в input (card field), названия
caption	12px	600 SemiBold	Дата, метаданные, метки
label	10px	700 Bold + uppercase	Input labels (ОТКУДА, КУДА), категории

```
// Next.js — подключение Nunito через next/font (self-hosted, нет GDPR проблем)
import { Nunito } from "next/font/google";
const nunito = Nunito({
  subsets: ["latin", "cyrillic"],
  weight: ["400", "600", "700", "800"],
  variable: "--font-nunito",
});
// Flutter
GoogleFonts.nunito(fontSize: 16, fontWeight: FontWeight.w700)
// Mini App (Vite)
// В index.html:
// <link href="https://fonts.googleapis.com/css2?family=Nunito:wght@400;600;700;800&subset=cyrillic" rel="stylesheet">
```

27.4 Иконки — Lucide

Библиотека	Платформа	Установка
lucide-react	Web + Mini App + Admin	npm install lucide-react
lucide_flutter	Flutter iOS/Android	lucide_flutter: ^0.x в pubspec.yaml

Ключевые иконки Kosh:

Иконка	Lucide название	Где используется
Точка на карте	MapPin	Откуда, точка отправления
Флаг/цель	Flag	Куда, точка прибытия
Стрела маршрута	ArrowRight	Разделитель в "Бишкек → Ош"
Пользователи	Users	Количество мест
Автомобиль	Car	Профиль водителя
Звезда	Star	Рейтинг (filled + empty)
Сообщение	MessageCircle	Чат, кнопка написать
Колокол	Bell	Уведомления (+ BellDot для непрочитанных)
Щит	ShieldCheck	Верифицированный водитель
Часы	Clock	Время отправления
Чемодан	Luggage	Багаж
Галочка	CheckCircle	Принято, успех
Крест	XCircle	Отклонено, ошибка
Предупреждение	AlertTriangle	Жалоба, предупреждение
Замок	Lock	Безопасность, скрытый телефон
Выход	LogOut	Кнопка выйти
Поиск	Search	Строка поиска
Настройки	Settings	Профиль, настройки
Камера	Camera	Загрузка фото документов

27.5 Input — Card Field стиль

Выбранный стиль: Card Field (вариант C)

Белый блок с border, иконка слева, uppercase label сверху, bold value снизу. При фокусе — border меняется на teal-500. Этот стиль выбран как основной для всех полей ввода на всех клиентах.

```
// React компонент — Card Field Input
interface CardFieldProps {
  icon: string;
  label: string;
  value?: string;
  placeholder?: string;
  onChange?: (v: string) => void;
}

// CSS классы (Tailwind)
const cardField = `
  flex items-center gap-3 bg-white border-[1.5px] border-gray-200
  rounded-2xl px-4 py-3 cursor-pointer
  focus-within:border-teal-500 transition-colors
`;

const iconWrap = "w-8 h-8 rounded-lg flex items-center justify-content-center text-base flex-shrink-0";
```

```

const labelCls = "text-[10px] font-bold text-gray-500 uppercase tracking-wider";
const valueCls = "text-sm font-bold text-gray-900 mt-0.5";
const inputCls = "text-sm font-bold text-gray-900 border-none outline-none bg-transparent w-full"
;

// Поисковая форма — 4 поля в одном блоке
<div className="border-[1.5px] border-gray-200 rounded-2xl overflow-hidden bg-white">
  <CardField icon="📍" label="ОТКУДА" value="Бишкек" iconBg="bg-teal-50" />
  <div className="border-t-[0.5px] border-gray-100" />
  <CardField icon="📍" label="КУДА" placeholder="Куда едете?" iconBg="bg-amber-50" />
  <div className="border-t-[0.5px] border-gray-100" />
  <div className="flex">
    <CardField icon="📅" label="ДАТА" value="20 апреля" iconBg="bg-teal-50" className="flex-1" />
    <div className="border-l-[0.5px] border-gray-100" />
    <CardField icon="👤" label="МЕСТ" value="1" iconBg="bg-gray-100" className="w-24" />
  </div>
  <div className="p-4">
    <SubmitButton>🔍 Найти поездку</SubmitButton>
  </div>
</div>

```

27.6 Кнопки

Токен	Цвет	Форма	Использование
btn-submit	Amber #F59E0B	rounded-xl, 13px 28px	ГЛАВНОЕ ДЕЙСТВИЕ: Найти поездку, Забронировать
btn-primary	Teal #0D9488	rounded-xl, 13px 28px	Принять запрос, Опубликовать, Подтвердить
btn-secondary	Teal-50 bg + teal border	rounded-xl	Отменить, Назад, Пропустить
btn-outline	White + teal border	rounded-xl	Написать водителю, Посмотреть профиль
btn-ghost	Gray-100 bg	rounded-xl	Второстепенные действия
btn-danger	Red-50 bg + red text	rounded-xl	Удалить, Заблокировать
btn-pill	Amber или Teal	rounded-full (pill)	СТА на лендинге, Hero кнопки

Правило Amber vs Teal

Amber = главное пользовательское действие (найти, забронировать, оплатить). Teal = операционное действие (принять, опубликовать, подтвердить). Никогда не используйте два Amber на одном экране.

27.7 Компоненты — карточки, бейджи, статусы

Компонент	Стиль	Использование
TripCard	white bg, 0.5px border-gray-300, rounded-2xl, p-14px	Карточка поездки в списке
DriverAvatar	Круг 32px, teal-100 bg, teal-700 текст, 12px Bold	Аватар по умолчанию (инициалы)
Badge verified	teal-100 bg, teal-800 text, rounded-full	✓ Верифицирован
Badge pending	amber-100 bg, amber-900 text, rounded-full	🕒 Ожидание
Badge seats	gray-100 bg, gray-700 text, rounded-full	N орун / N мест

Компонент	Стиль	Использование
Status dot accepted	green dot + green-50 bg	Принято — в деталях брони
Status dot pending	amber dot + amber-50 bg	Ожидание — в деталях брони
Status dot rejected	red dot + red-50 bg	Отклонено — в деталях брони
NotifCard	teal-50 bg, border-left 3px teal-500, rounded-xl	Успешные уведомления
NotifCard warning	amber-50 bg, border-left 3px amber-500, rounded-xl	Напоминания
ProgressBar	gray-200 track, teal-500→teal-400 fill, 6px height, pill	Прогресс верификации
RatingStar	amber-500 filled, gray-300 empty, 14px	Рейтинг водителя/пассажира
BottomNav	white bg, border-top 0.5px, 4 вкладки, active = teal-600	Мобильная навигация
TopNav	white bg, border-bottom 0.5px, logo + search + avatar	Desktop навигация

27.8 Отступы и скругления

Токен	Значение	Использование
space-1	4px	Минимальный гар между inline элементами
space-2	8px	Гар между иконкой и текстом
space-3	12px	Padding карточки (compact), гар в row
space-4	16px	Padding карточки (default), гар между секциями
space-6	24px	Padding страницы (mobile), гар между карточками
space-8	32px	Padding страницы (desktop)
radius-sm	8px (rounded-lg)	Бейджи, теги, маленькие элементы
radius-md	12px (rounded-xl)	Кнопки, inputs, маленькие карточки
radius-lg	16px (rounded-2xl)	Основные карточки, поисковая форма
radius-xl	20px (rounded-3xl)	Модальные окна, большие блоки
radius-full	9999px (rounded-full)	Pill кнопки, аватары, бейджи

27.9 Breakpoints — responsive система

Breakpoint	Tailwind prefix	Ширина	Применение
mobile (default)	нет префикса	< 640px	Базовый — одна колонка, bottom nav
sm	sm:	≥ 640px	Небольшие корректировки padding
md	md:	≥ 768px	Две колонки в поиске, sidebar появляется
lg	lg:	≥ 1024px	Три колонки (split view), top nav вместо bottom
xl	xl:	≥ 1280px	Увеличенный контейнер (max-w-7xl)

Mobile-first обязательно

Весь CSS пишется от mobile к desktop: сначала базовый стиль (< 640px), потом md: и lg: модификаторы. Никогда не наоборот.

27.10 Тёмная тема — Этап 2

В MVP — только светлая тема. Tailwind dark: классы добавляются в Этапе 2. Подготовка уже в MVP: использовать CSS переменные для цветов через Tailwind config, не хардкодить hex. Это позволит добавить dark mode за один спринт.

26.1 Стек и архитектура

Слой	Библиотека	Назначение
State management	Riverpod	Типобезопасный provider-based подход
Routing	go_router	Декларативная навигация, deep links
HTTP	dio + retrofit	REST клиент с кодогенерацией из API
WebSocket	socket_io_client	Тот же протокол что Mini App
Локальная БД	Hive	Offline queue + кэш
Secure storage	flutter_secure_storage	Токены в Keychain/Keystore
Локализация	easy_localization	JSON файлы ru/ky
Формы	reactive_forms	Валидация аналогично react-hook-form
Карты	flutter_map (Leaflet-compatible)	OpenStreetMap тайлы
Push	firebase_messaging	FCM (Этап 2)
Image picker	image_picker + image_cropper	Фото документов
Image compression	flutter_image_compress	Перед загрузкой на сервер

26.2 Требования к платформам

Платформа	Минимальная версия	Целевая
iOS	13.0	17+
Android	6.0 (API 23)	14 (API 34)
Screen sizes	360×640	Адаптивный дизайн

26.3 Публикация

App Store: платный developer account (\$99/год), двухэтапная публикация через App Store Connect, модерация 1–7 дней. Google Play: разовая оплата (\$25), модерация до 3 дней. План: публикация на Google Play первой (быстрее), App Store через 2 недели.

Категория	Требование	Метрика/проверка
Производительность	API p95 latency	< 300 ms при 100 RPS
Производительность	API p99 latency	< 800 ms при 100 RPS
Производительность	Mini App bundle size	< 500 KB gzipped
Производительность	Mini App FCP (First Contentful Paint)	< 1.5 s на 4G
Производительность	Flutter app размер установки	< 25 MB
Производительность	Flutter app cold start	< 3 s на средних устройствах
Доступность	Uptime	99.5% (42 минуты даунтайма в месяц)
Доступность	Plan maintenance window	Воскресенье 03:00–05:00 UTC+6
Масштабируемость	Concurrent users	1000 (MVP), 10 000 (Этап 2)
Масштабируемость	DB size	10 GB (MVP), 100 GB (Этап 2)
Безопасность	HTTPS everywhere	Нет HTTP, редирект на HTTPS
Безопасность	OWASP Top 10	Проверка через automated scan раз в месяц
Безопасность	Passwords (admin)	bcrypt rounds=12
Безопасность	Секреты	Environment variables, не в коде, не в Git
Надёжность	Graceful shutdown	Корректное завершение при SIGTERM (15 сек)
Надёжность	Health check	GET /health проверяет БД + Redis + диск
Наблюдаемость	Structured logs	JSON, поля: level, msg, request_id, user_id, duration
Наблюдаемость	Сохранение логов	30 дней, ротация logrotate
Наблюдаемость	Метрики	Uptime Kuma проверяет /health каждые 30 сек
Наблюдаемость	Алерты	Error rate > 1% за 5 минут → Telegram DevOps
Удобство для разработчиков	API docs	OpenAPI 3.0 Swagger на /docs (не прод)
Удобство для разработчиков	Database seeds	Скрипт для создания тестовых данных
Совместимость	Browser support (Mini App)	Telegram WebView: Chrome 90+, Safari 14+
Совместимость	Web browser support	Chrome/Firefox/Safari последние 2 версии
Поддержка	Response time поддержка	Критично: 1 час. Обычно: 24 часа.

Обзор этапов

Этап	Название	Фокус	Метрика входа	Метрика выхода
Этап 1	MVP	Базовый продукт — поиск, бронирование, чат, рейтинги	Старт	500 DAU · 200 водителей
Этап 2	Рост	Карты, трекинг, push, offline, новые маршруты	500 DAU	2 000 DAU · 5 маршрутов
Этап 3	Зрелость	Посылки, корпоративные, программа лояльности, МВД	2 000 DAU	10 000 DAU · 10+ маршрутов

Этап 1 — MVP (недели 1-14)

Команда MVP

Роль	Кол-во	Зона ответственности
Backend Engineer (Senior)	1	API, БД, WebSocket, интеграции, Auth
Backend Engineer (Middle)	1	Admin API, миграции, cron, тесты
Flutter Developer (Middle+)	1	iOS/Android приложение
Frontend Developer (Senior)	1	Mini App + Web + Admin Panel
QA Engineer	0.5	Ручное тестирование, автоматизация
DevOps / SRE	0.5	Инфраструктура, CI/CD, мониторинг
Product Owner	1	Координация, приоритизация
Переводчик (ky)	Freelance	Перевод интерфейса на кыргызский

Спринты

Спринт	Недел и	Цель	Демо
Sprint 0 — Подготовка	1	Сервер, CI/CD, скелет, миграции БД	Деплой-пайплайн работает
Sprint 1 — Auth	2-3	Auth (Telegram + phone + Google), профиль	Онбординг пассажира
Sprint 2 — Верификация	4-5	Верификация водителей, Admin базово	Админ верифицирует
Sprint 3 — Поездки	6-7	Публикация, поиск, детали	Водитель публикует, пассажир ищет
Sprint 4 — Бронирование + Чат	8-9	Бронирование, pre-booking чат, статусы RT	Полный цикл с чатом
Sprint 5 — Рейтинги + Уведомления	10	Рейтинги, Telegram Bot, все уведомления	Оценки, уведомления
Sprint 6 — Web + Полировка	11-12	Web клиент, жалобы, дашборд, баги	Beta ready

Спринт	Недел и	Цель	Demo
Beta testing	13	10 водителей, 50 пассажиров	Feedback loop
Soft launch	14	Публичный запуск на 3 маршрутах	Первые реальные поездки

Инфраструктура Этап 1

Компонент	Решение	Стоимость/мес
Сервер	DigitalOcean / Hetzner — 4 vCPU, 8GB RAM	\$40–60
БД	PostgreSQL 16 на том же сервере	включено
Файлы	Локальный диск + Nginx	включено
SSL	Let's Encrypt + Certbot	бесплатно
Мониторинг	Uptime Kuma (self-hosted)	бесплатно
SMS	Mega.kg или Nikita (КГ провайдер)	\$10–30
Итого	—	\$50–90/мес

Этап 2 — Рост (месяцы 4–9)

Условие входа в Этап 2

500+ DAU стабильно 2 недели подряд · 200+ верифицированных водителей · NPS > 40 по итогам beta · retention Day-7 > 30%

Функциональность Этапа 2

Модуль	Фиичи	Приоритет
Push уведомления	FCM для Flutter iOS/Android. Web Push через Service Worker.	P0
Карта маршрута	Реальные координаты остановок. PostGIS для geospatial queries. Leaflet карта маршрута на всех клиентах.	P0
Трекинг поездки	Водитель шарит геолокацию при старте. Пассажир видит где водитель в реальном времени. Этапы: ожидание → в пути → прибыл.	P0
Offline Flutter	Полноценная Hive offline queue. Синхронизация при восстановлении. Кэш поездок на 24 часа.	P1
Redis	Socket.IO Adapter для масштабирования. Rate limiting через Redis. Кэш популярных маршрутов.	P1
Расширение маршрутов	Добавляем: Бишкек↔Джалал-Абад, Бишкек↔Талас, Бишкек↔Чолпон-Ата, Ош↔Джалал-Абад.	P1
Web Push	Service Worker + FCM для Web клиента. Уведомления без открытого браузера.	P1
Реферальная программа	Реферальный код при регистрации. +10% поездок за приглашённого друга (скидка на будущие функции).	P2
Расширенная статистика водителя	График доходов по неделям. Топ маршруты. Процент принятия. Сравнение с средним по платформе.	P2

Модуль	Фичи	Приоритет
Instant booking режим	Водитель выбирает при публикации: "Я выбираю пассажиров" или "Мгновенное бронирование". В instant режиме — пассажир едет без подтверждения.	P2
Apple Sign In	Обязателен для iOS App Store (правило Apple). Уже в стеке Auth.	P0
Android / iOS публикация	Google Play + App Store. Прохождение ревью. ASO оптимизация.	P0

Команда Этапа 2

Роль	Изменение
Backend Engineer (Senior)	Остаётся
Backend Engineer (Middle)	Остаётся
Flutter Developer (Middle+)	Остаётся + трекинг геолокации
Frontend Developer (Senior)	Остаётся + Web Push
QA Engineer	0.5 → 1.0 (полная ставка)
DevOps / SRE	0.5 → 1.0 (Redis, масштабирование)
Mobile Developer (iOS)	+1 новый (App Store специфика, Swift если нужно)
Data Analyst	+0.5 новый (аналитика поведения)

Инфраструктура Этапа 2

Компонент	Решение	Стоимость/мес
Сервер API	DigitalOcean — 8 vCPU, 16GB RAM	\$80-120
Сервер WebSocket	Отдельный инстанс — 4 vCPU, 8GB	\$40-60
БД	Managed PostgreSQL (DO/Hetzner) + PostGIS	\$30-50
Redis	Managed Redis (DO) или self-hosted	\$15-30
Файлы	S3-совместимое (Spaces DO или Backblaze B2)	\$10-20
CDN	Cloudflare Free tier	бесплатно
Мониторинг	Uptime Kuma + Grafana/Prometheus	\$10-20
Итого	—	\$185-300/мес

Этап 3 — Зрелость (месяцы 10-18)

Условие входа в Этап 3

2 000+ DAU · 5+ активных маршрутов · retention Day-30 > 20% · NPS > 50 · iOS и Android приложения опубликованы и работают стабильно

Функциональность Этапа 3

Модуль	Фичи	Приоритет
Доставка посылок	Отдельный флоу: отправитель создаёт посылку, водитель берёт по пути. Размер, вес, фото. Получатель подтверждает получение. Рейтинг отдельный от поездок.	P0

Модуль	Фи́чи	Приоритет
Корпоративные аккаунты	Компания регистрирует аккаунт. Добавляет сотрудников. Консолидированная история поездок. Отчёты для бухгалтерии.	P0
Интеграция с МВД КГ	Автоматическая проверка водительских прав по базе МВД. Ускоряет верификацию — с 24ч до нескольких минут.	P1
Программа лояльности	Баллы за поездки (1 поездка = 10 баллов). Статусы: Новичок, Путешественник, Эксперт, Элита. Привилегии по статусу: приоритет в поиске, бейдж на профиле.	P1
Аналитика поведения	Heatmap популярных маршрутов. Воронка отмен (где теряем). A/B тесты UX элементов. Cohort retention анализ.	P1
Расширенный поиск	Поиск по промежуточным остановкам. "Еду через Токмок — есть пассажиры?" Radius search когда появится PostGIS.	P2
SOS кнопка	Экстренная кнопка в активной поездке. Отправляет геолокацию + уведомление доверенному контакту. Алерт администратору.	P1
Мультиязычность URL	kosho.kg/ky/trips — кыргызский язык в URL для SEO.	P2
API для агрегаторов	Публичное API для партнёров (не монетизационное — только данные о маршрутах и расписании).	P3
Тёмная тема	Dark mode на всех клиентах через Tailwind dark: и Flutter ThemeData.	P2

Команда Этапа 3

Роль	Изменение
Backend Engineers	2 → 3 (добавляем для посылок и корп. аккаунтов)
Flutter Developer	1 → 2 (iOS специалист отдельно)
Frontend Developer	1 → 2 (выделяем Web разработчика отдельно)
QA Engineer	1 → 1.5
DevOps / SRE	1 → 1 (масштабирование уже настроено)
Data Analyst	0.5 → 1.0
UX Designer	+1 новый (для A/B тестов и улучшения конверсии)
Customer Support	+1 новый (жалобы, верификация, поддержка)

Инфраструктура Этапа 3

Компонент	Решение	Стоимость/мес
API серверы	2-3 инстанса с load balancer	\$200-300
WebSocket	2 инстанса + Redis Adapter	\$100-150
БД	Managed PostgreSQL — primary + replica	\$100-150
Redis	Managed Redis Cluster	\$50-80
Файлы	S3 + CDN (Cloudflare R2)	\$30-50
Аналитика	ClickHouse для событий (self-hosted)	\$50-80

Компонент	Решение	Стоимость/мес
Мониторинг	Grafana Cloud или self-hosted	\$20-40
Итого	—	\$550-850/мес

30.1 Definition of Done для фичи

Чеклист	Обязательно
Написан и прошёл code review	☐
Unit тесты покрывают основные сценарии (>70%)	☐
Integration тест проходит в CI	☐
Нет ошибок в логах на staging	☐
TypeScript strict, без any	☐
ru.json + ky.json обновлены	☐
Swagger документация обновлена	☐
QA подтвердил на staging	☐
Нет regression багов	☐

Формат: Given (исходное состояние) / When (действие) / Then (ожидаемый результат).

ID	Given / When / Then	Метод проверки
AC-01	G: Новый пользователь, открывает Mini App через Telegram. W: Telegram передаёт initData. T: Создаётся user, выдаётся JWT, видит экран ввода телефона.	E2E Cypress + ручное на реальном TG
AC-02	G: Пользователь с неverified телефоном. W: Вводит +996XXX и жмёт "Отправить код". T: Получает SMS за <30 сек. В БД — запись otp_codes с хешем.	Ручное с реальным номером
AC-03	G: OTP отправлен. W: Вводит неверный код 5 раз. T: На 6-й попытке ошибка "Слишком много попыток. Попробуйте через 15 минут."	Integration test
AC-04	G: Verified пассажир. W: Выбирает роль "Водитель" и загружает 4 фото. T: driver_profile создаётся со status=pending. Админ видит заявку.	E2E
AC-05	G: Админ в очереди верификации. W: Одобряет заявку. T: status=verified, у user.roles появляется "driver", водитель получает уведомление в TG Bot за <5 сек.	E2E + ручное
AC-06	G: Verified водитель. W: Создаёт поездку Бишкек→Ош, 3 места, 800 сом. T: В поиске этой поездки другой пользователь видит её в течение 2 секунд.	E2E
AC-07	G: Поездка с seats_available=1. W: Два пассажира одновременно отправляют POST /bookings. T: Ровно один получает 201, второй — 409 Conflict. В БД seats_available корректно.	Load test (k6) + проверка логов
AC-08	G: Водитель получил booking request. W: Жмёт "Принять". T: Пассажир получает уведомление за <5 сек. Чат открывается. seats_available уменьшается.	E2E + timing test
AC-09	G: Booking со status=accepted. W: Пассажир отправляет сообщение в чат. T: Водитель видит сообщение в другой сессии за <2 сек через WebSocket.	E2E с двумя сессиями
AC-10	G: Booking со status=pending, прошёл 1 час без ответа. W: Cron задача запускается. T: status=expired, пассажир получает уведомление.	Integration test с подменой времени
AC-11	G: Поездка прошла (departure + duration + 2h). W: Cron запускается. T: trip.status=completed, обе стороны получают push "оцените поездку".	Integration test
AC-12	G: Обе стороны оценили. W: Insert в ratings. T: users.rating пересчитан, если <4.0 — уведомление водителю.	Unit + integration
AC-13	G: Любой эндпоинт. W: 100 параллельных запросов через k6. T: p95 latency < 300 ms, error rate < 0.1%.	Load test
AC-14	G: Slow 3G эмуляция в Chrome DevTools. W: Открываем Mini App. T: FCP < 2 сек, TTI < 3 сек.	Lighthouse
AC-15	G: Пользователь оффлайн. W: Пишет сообщение в чат. T: Сообщение ставится в очередь, при восстановлении связи отправляется. UI показывает статус "Отправляется".	Manual offline test
AC-16	G: Access token истёк. W: Клиент отправляет запрос. T: Получает 401, автоматически refresh, повторяет запрос — успешно.	Integration
AC-17	G: Пассажир с накопленной историей. W: DELETE /users/me. T: users.deleted_at установлен, phone обнулён, через 30 дней физически удалён.	Unit + проверка cron

ID	Given / When / Then	Метод проверки
AC-18	G: Админ подаёт жалобу P0 (безопасность). W: Создаётся complaint. T: Суперадмин получает SMS за 1 минуту.	Integration
AC-19	G: Язык пользователя = ky. W: Получает любое уведомление от бота. T: Текст на кыргызском языке.	Ручное + integration
AC-20	G: Mini App загружается. W: Открываем в Telegram WebView на iPhone SE (минимальное устройство). T: Всё читаемо, нет горизонтального скrolла, все кнопки тапабельны.	Ручное на реальном устройстве

Риск	Вероятность	Воздействие	Митигация
Cold start: нет водителей, нет пассажиров	Высокая	Критичное	Ручной рекрутинг первых 20 водителей. Бесплатные поездки первую неделю для пассажиров.
Мошенничество: фейковые водители с поддельными документами	Средняя	Высокое	Строгая ручная верификация. Селфи + фото в правах должны совпадать. В будущем — интеграция с API МВД.
Конкурент (inDrive) сделает локализацию под КГ быстрее	Средняя	Высокое	Скорость запуска. MVP за 10 недель. Гиперлокальность: Telegram Mini App даёт преимущество.
Регуляторный риск: могут признать коммерческим такси	Низкая	Критичное	Юридическая консультация до запуска. Позиционирование как "деление расходов" (cost-sharing).
DDoS или взлом	Низкая	Высокое	Cloudflare на проде. Регулярный security audit. Бэкапы.
Ухудшение работы Telegram в КГ (блокировки)	Низкая	Критичное	Дублирование всего функционала в Flutter приложении с Этапа 2.
Потеря данных (БД corruption, человеческая ошибка)	Низкая	Критичное	Ежедневные бэкапы, тестирование восстановления раз в месяц.
Сбой SMS провайдера	Средняя	Среднее	Два провайдера: основной + резерв. Автоматическое переключение.
Неготовность водителей к цифровому продукту	Высокая	Среднее	Упрощённый UX. Обучающие видео. Саппорт в Telegram канале.
Убытки на SMS (злоупотребления)	Средняя	Среднее	Rate limit: 5 SMS в сутки на номер. Мониторинг расходов еженедельно.
Текучка водителей после первых проблем	Высокая	Высокое	Быстрая реакция на жалобы. Поддержка. Retention-программы (бейджи, статусы).
Технический долг из-за скорости MVP	Высокая	Среднее	Спринт 6 — неделя на полировку. Код-ревью. Тесты с начала.

Подпись и утверждение

Документ	Kosho — T3 Платформа попутчиков Кыргызстан
Версия	2.3.0 — Pre-booking Chat + Full Roadmap
Дата	April 2026
Стек	Node.js + Express + TypeScript · PostgreSQL 16 · Next.js · Vite + React · Flutter
Клиенты	Telegram Mini App · Flutter iOS/Android · Web · Admin Panel · Telegram Bot
Design System	Nunito · Teal #0D9488 · Amber #F59E0B · Lucide Icons
Команда MVP	5.0 FTE (2 BE + 1 Flutter + 1 FE + 0.5 QA + 0.5 DevOps)
MVP срок	10-12 недель разработки + 2 недели beta
Бюджет инфраструктуры	\$40-60/мес (MVP) → \$120-200/мес (после 1000 DAU)
Статус	Готов к передаче в разработку
Следующий шаг	Kick-off meeting, назначение тим-лида, создание проекта в GitLab

Changelog v2.1 → v2.2

Изменение	Причина
[v2.2] Раздел 26 — Web приложение (полный)	18 страниц, breakpoints, SEO, sitemap, canvas compress, wizard
[v2.2] Раздел 27 — Design System	Цвета Teal & Amber, Nunito, Lucide, Input C, Amber button
[v2.2] Input стиль — Card Field	Выбран: белый блок, uppercase label, bold value, icon слева
[v2.2] Submit кнопка — Amber #F59E0B	Выделяется на фоне teal, тёплый акцент
[v2.2] Шрифт — Nunito вместо Plus Jakarta Sans	Plus Jakarta Sans не поддерживает кириллицу
[v2.2] Web стек детализирован	Next.js 14, next-intl, react-leaflet, shadcn/ui
[v2.2] Нумерация разделов 27-30 → 29-32	Вставка Web и Design System разделов
[v2.1] Auth — Google, Apple, Phone+Password	Расширение провайдеров входа
[v2.1] Deferred Action	sessionStorage, TTL 15 мин, 5 действий
[v2.1] Token Reuse Detection	Инвалидация всех сессий при краже токена
[v2.1] Видимость телефона по статусам booking	Защита от обхода платформы
Backend на Express.js (не Next.js)	Next.js не подходит для Socket.IO + cron
Mini App на Vite + React	Бандл в 3× меньше, критично для 4G
SELECT FOR UPDATE	Race condition на seats_available
estimated_duration_min	Автозаккрытие по реальной длительности маршрута
6 новых таблиц БД	otp_codes, refresh_tokens, complaints, cities, admin_actions, notifications